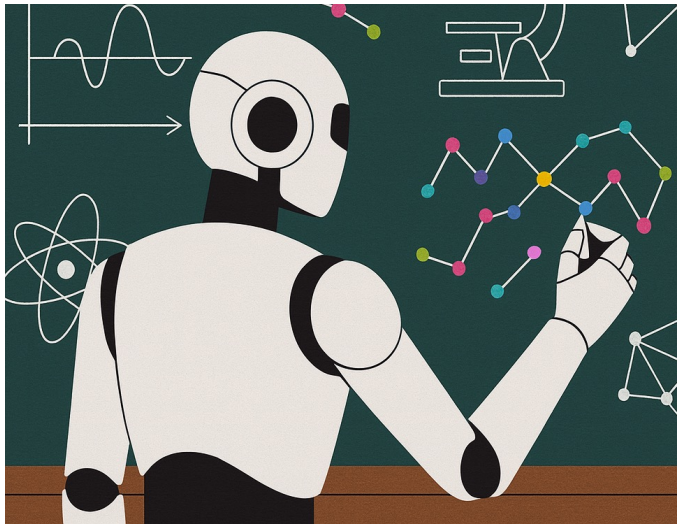
AI for (Quantum) Physics

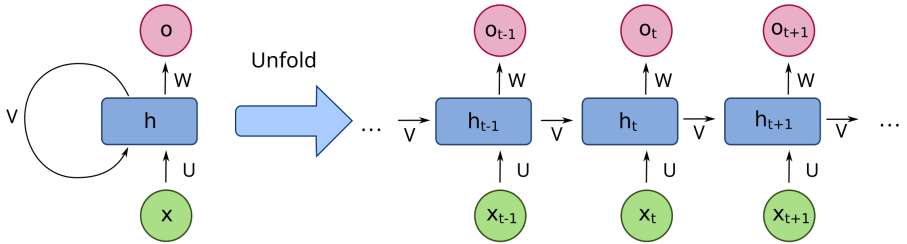


Xuemei Gu
xuemei.gu@uni-jena.de

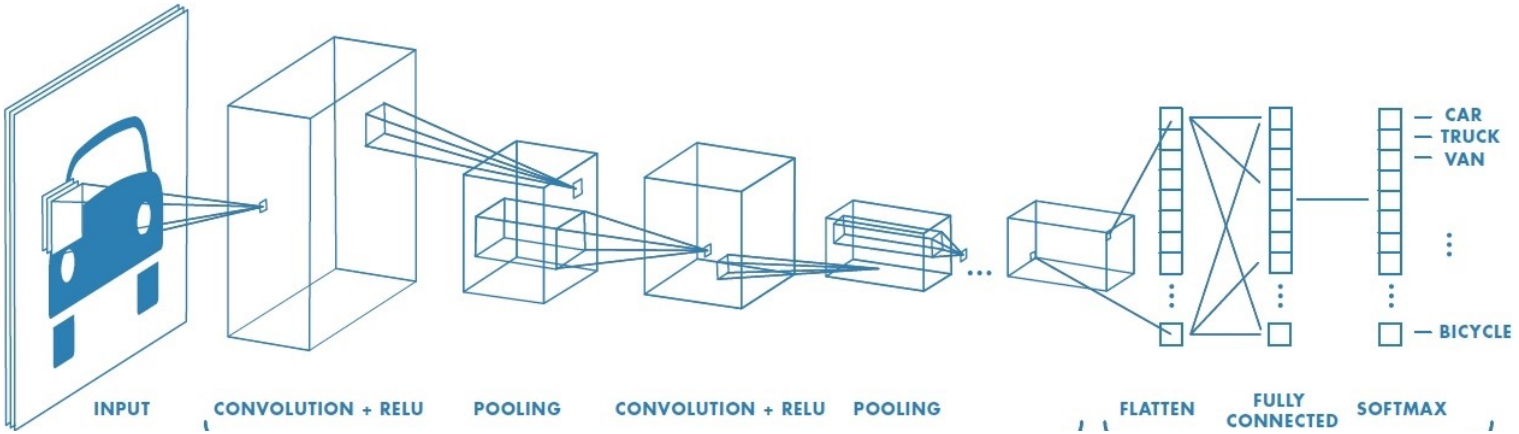
If you have any questions, feel free to reach out!

Today's topic:

- Recurrent neural network
- RNN variants
- Applications



Convolutional Neural Network



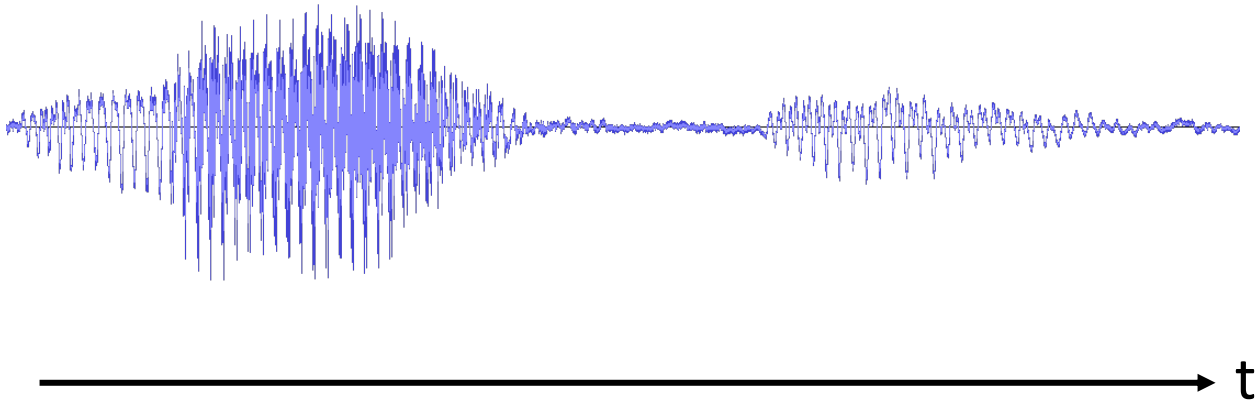


CNN

- Object classes
- Object detection
- Segmentation
- etc.

Sequence data

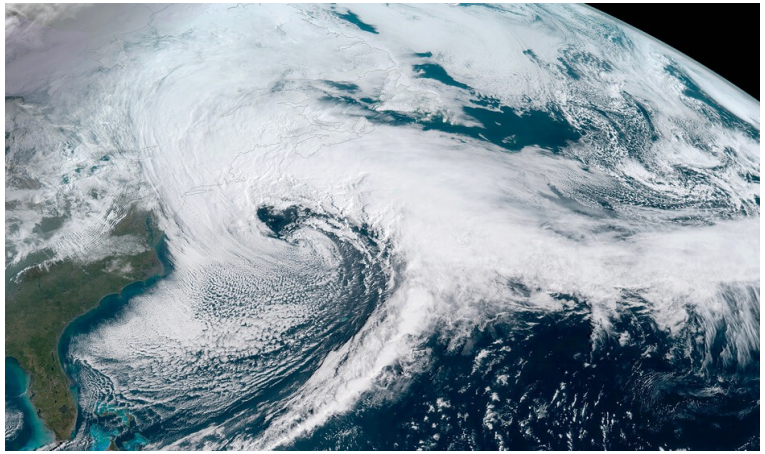
Data depends on time/order



Hello, nice to meet you!

Sequence data





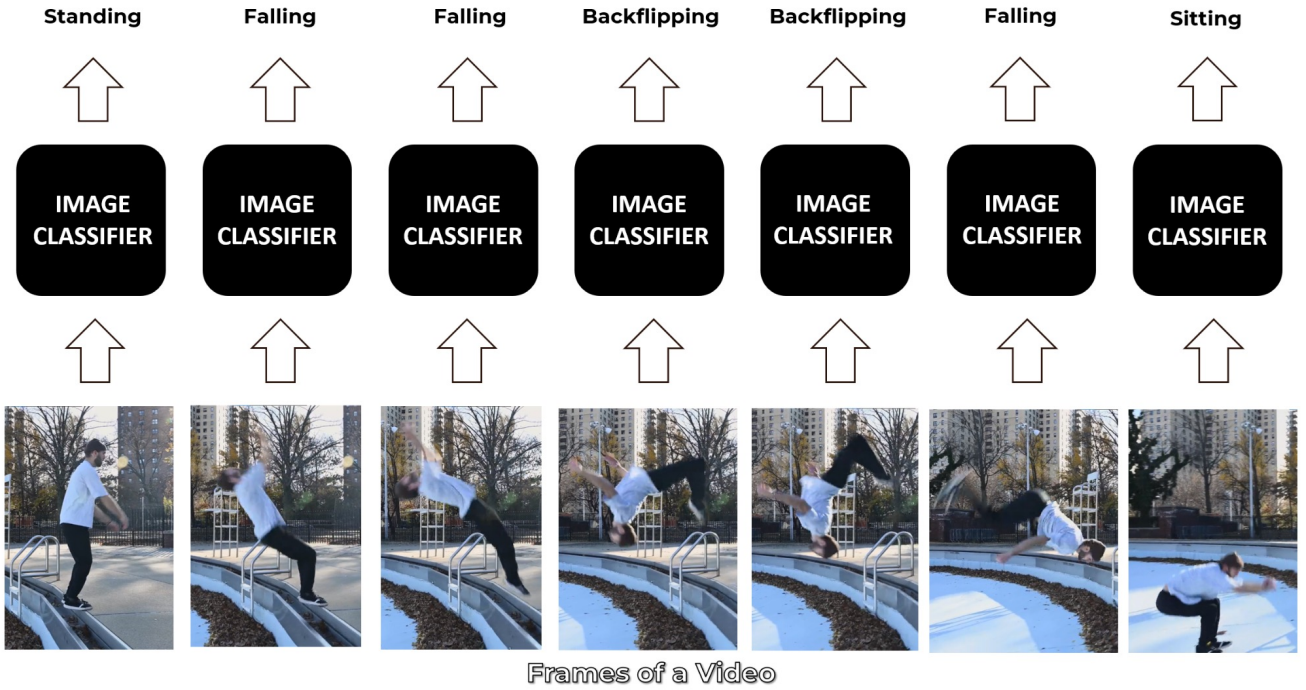






Sequence data



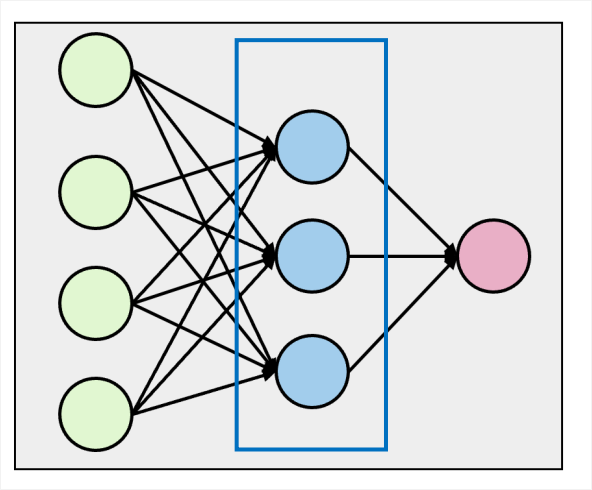


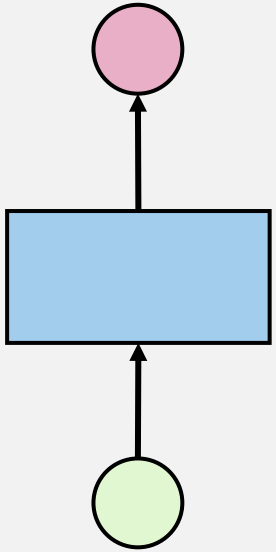
https://bleedaiacademy.com/human-activity-recognition-using-tensorflow-cnn-lstm/

Recurrent neural network

Each layer receives input from the previous layer and the output from the previous time step

Feedforward Neural Networks

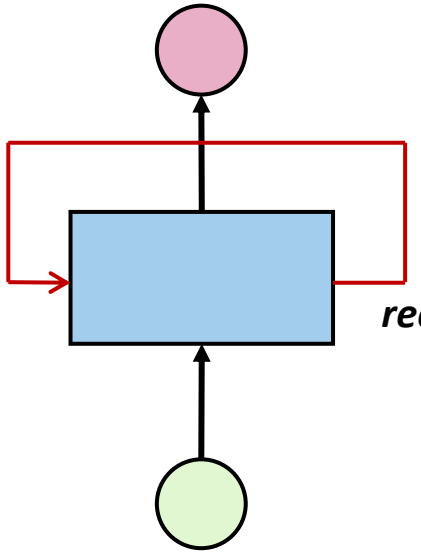




no memory of past since weights learned independently

Each layer receives input from the previous layer with no loops

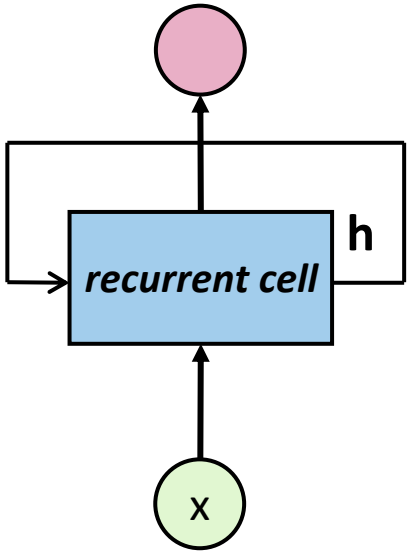
Key idea:
use hidden state to capture information about the past



recurrent cell memory

Recurrent neural network

One can process a sequence of vectors x by applying a **recurrence formula** at every time step:



State Update

$$h_t = f_W(h_{t-1}, x_t)$$

new state

old state

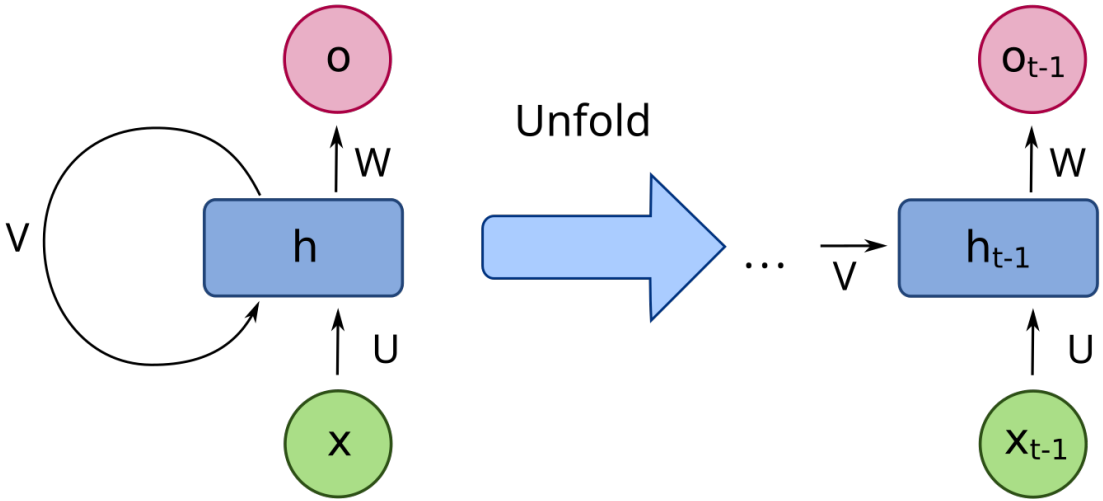
Some function with parameters W

Input vector at some time step

the same function and the same set of parameters are used at every time step

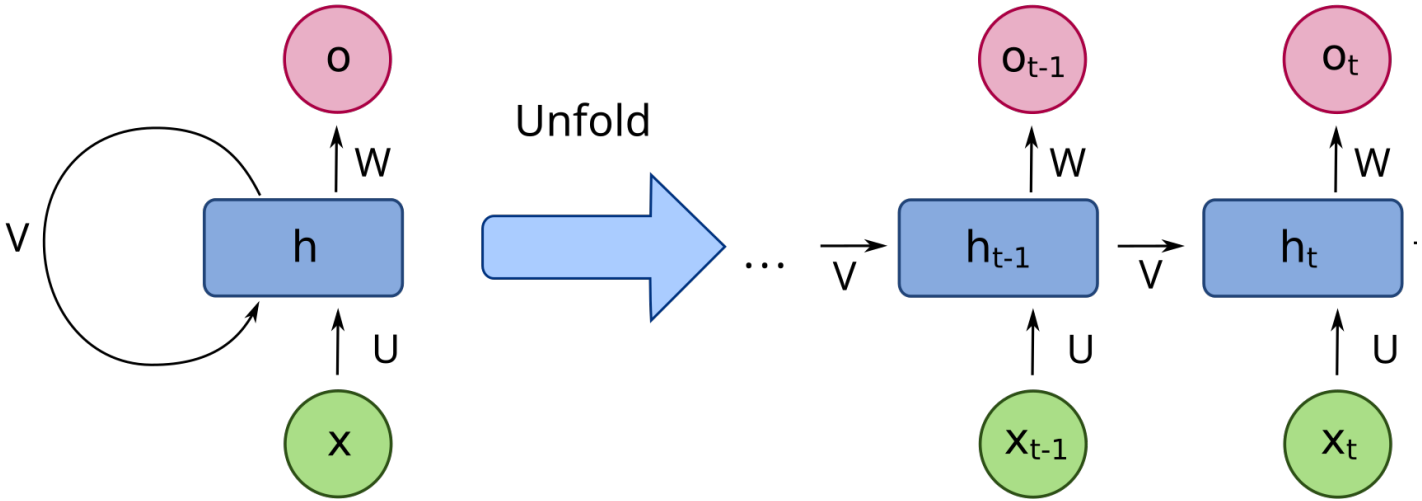
Recurrent neural network

RNN: Time Step 1



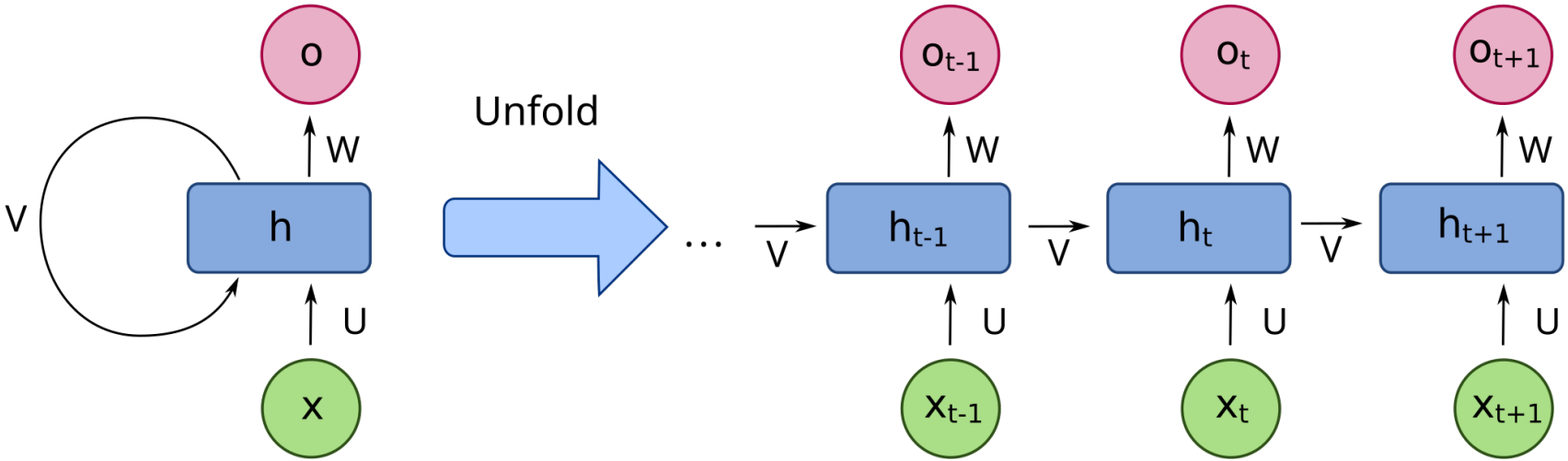
Recurrent neural network

RNN: Time Step 2



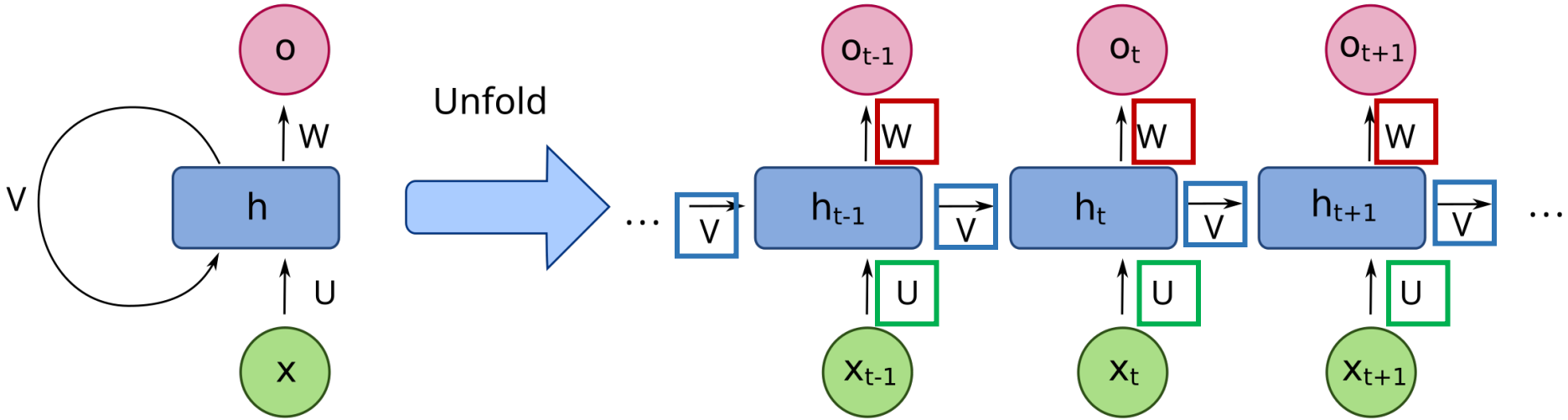
Recurrent neural network

RNN: Time Step 3



Recurrent neural network

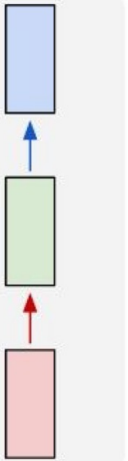
RNN: And So On…



- Overcomes problem that weights of each layer are learned independently by using previous hidden state
- Overcomes problem that model has many parameters since weights are shared across layers
- Retains information about past inputs for an amount of time that depends on the model's weights and input data rather than a fixed duration selected a priori

Types of Recurrent Neural Networks

one to one

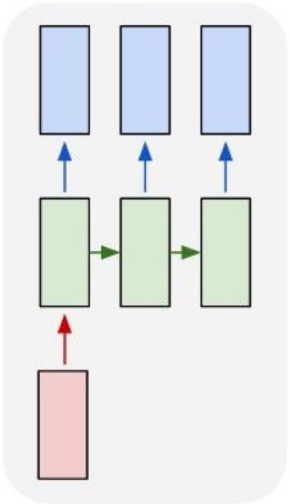


The model takes a fixed size input and produces a fixed size output, such as CNN, FNN.

Types of Recurrent Neural Networks

one to one

one to many

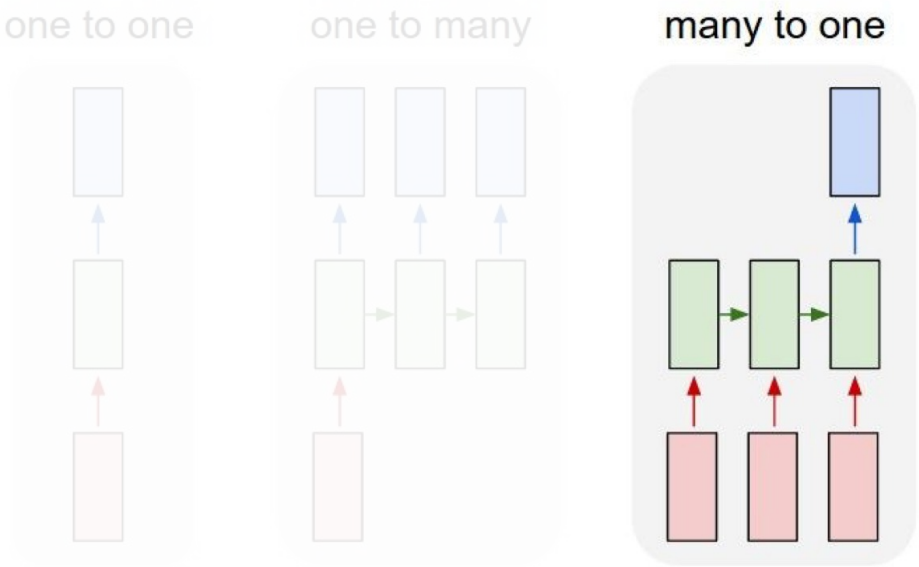


- Input: fixed-size
- Output: sequence
- e.g., image captioning

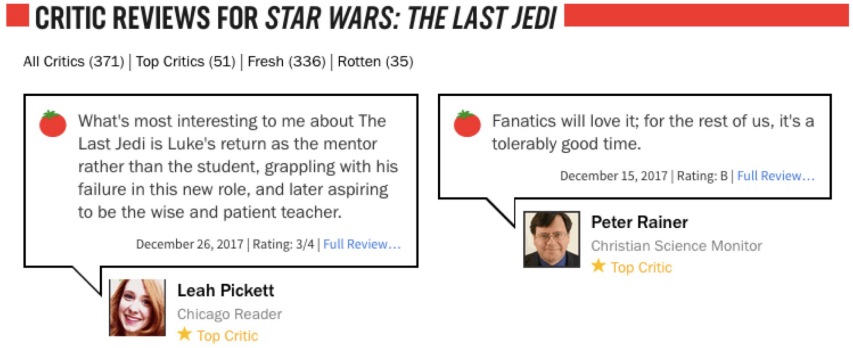




Types of Recurrent Neural Networks



- Input: sequence
- Output: fixed-size
- e.g., sentiment analysis (hate? love?, etc)



"This movie is fantastic! I really like it because it is so good!" ★★★★☆

"Not to my taste, will skip and watch another movie" ★★☆☆☆

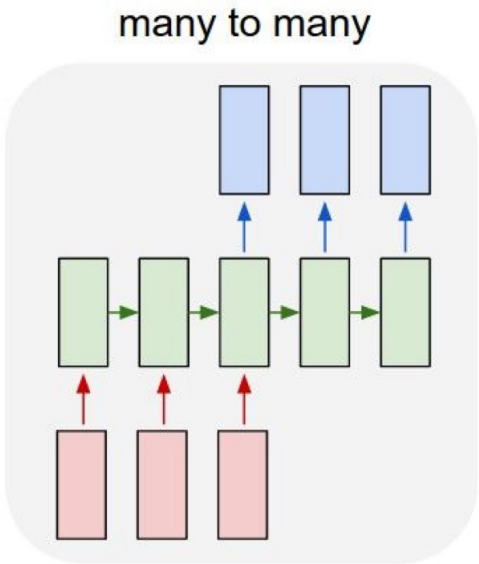
"This movie really sucks! Can I get my money back please?" ★☆☆☆☆

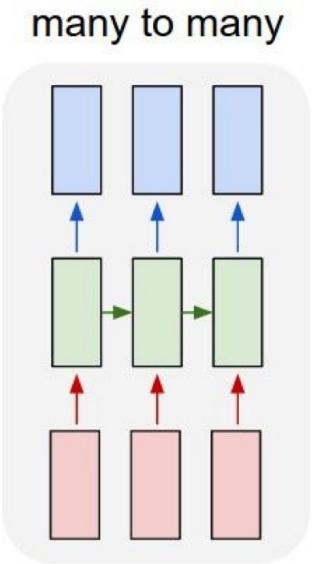
Types of Recurrent Neural Networks

- Input: sequence
- Output: sequence
- e.g., language translation, video captioning, video classification on frame level

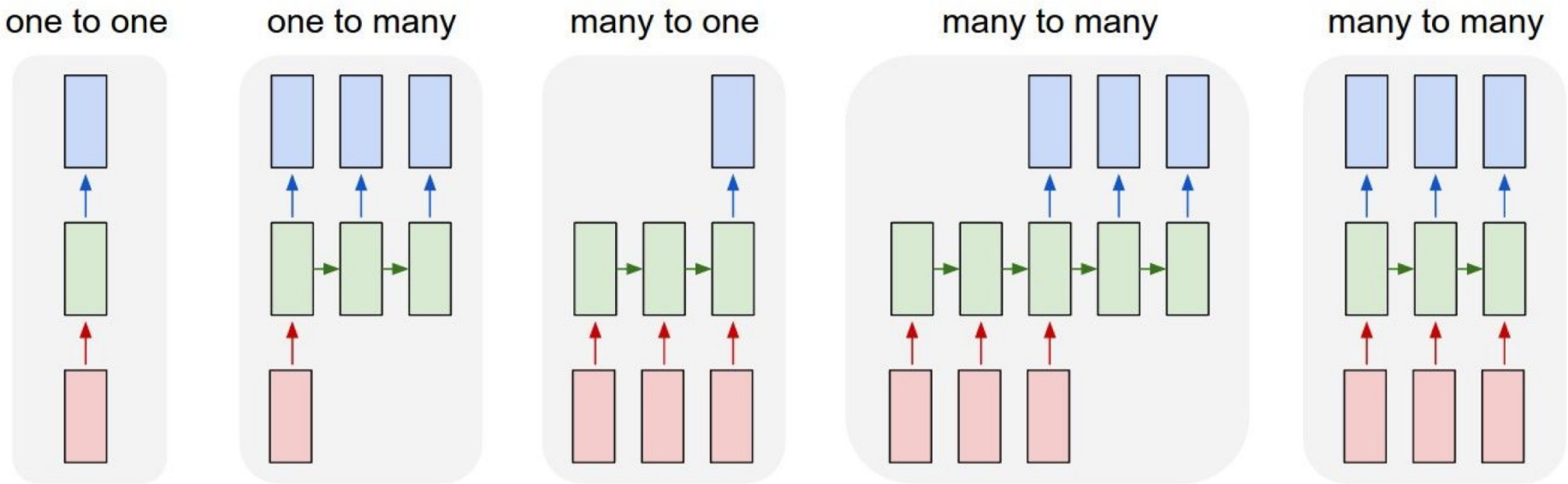








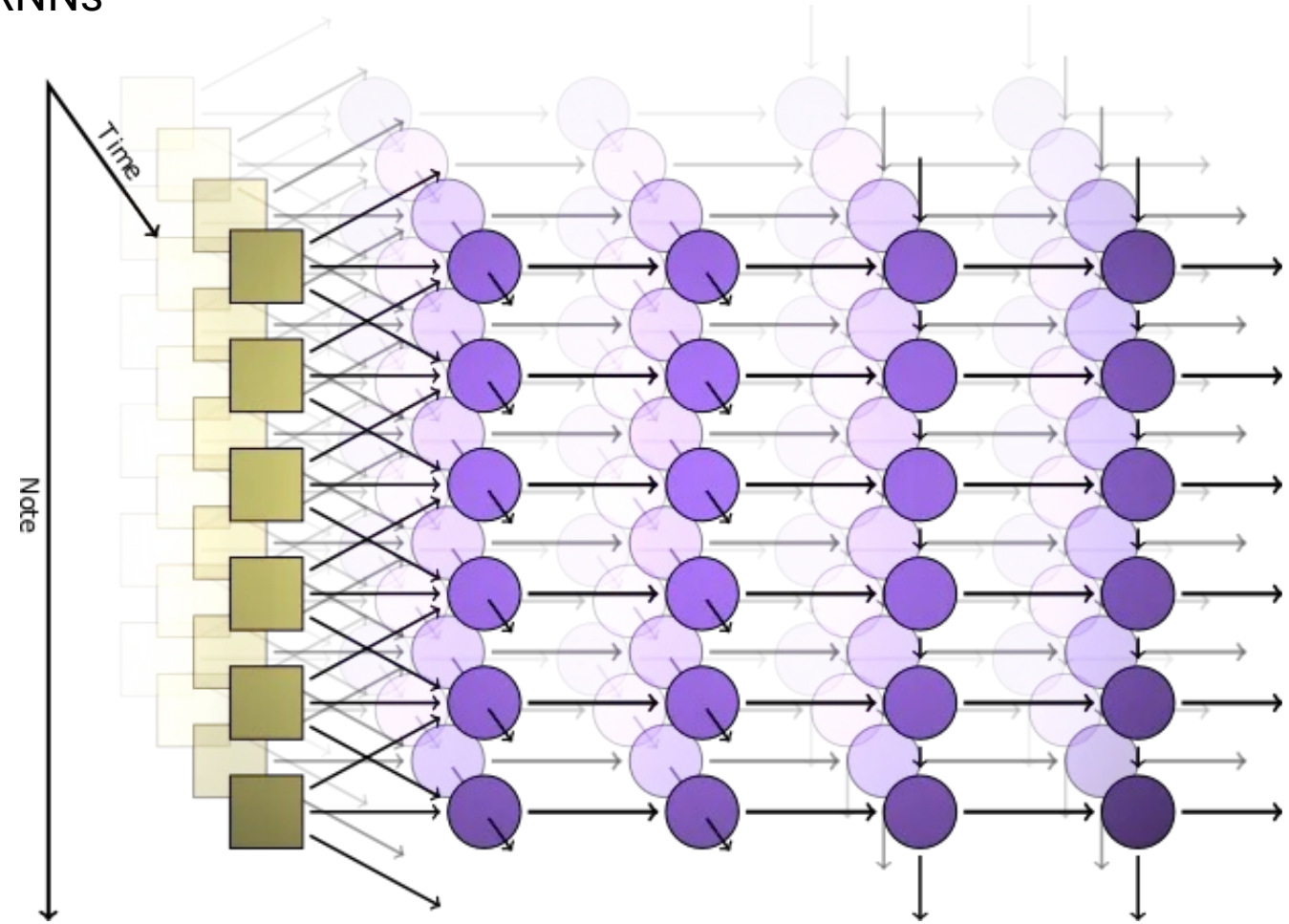
Types of Recurrent Neural Networks



Sequence Applications

1. What are other examples of “one-to-many” applications?
2. What are other examples of “many-to-one” applications?
3. What are other examples of “many-to-many” applications?

Multilayer RNNs



```python
class RNNSequenceModel(nn.Module):
    """
    RNN that processes the sequence step by step.
    Keeps a hidden state over time.
    """
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            batch_first=True  # input: [batch, seq_len, input_size]
        )
        self.fc_out = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        """
        x: [batch, seq_len, input_size]
        """
        # rnn_out: [batch, seq_len, hidden_size]
        rnn_out, h_n = self.rnn(x)

        # apply a linear layer at every time step
        y = self.fc_out(rnn_out)  # [batch, seq_len, output_size]
        return y
```

```python
class ManualRNNSequenceModel(nn.Module):
    """
    Same as RNNSequenceModel, but recurrence is written explicitly.
    """
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.hidden_size = hidden_size

        # Parameters for h_t = tanh(W_xh x_t + W_hh h_{t-1} + b_h)
        self.W_xh = nn.Linear(input_size, hidden_size, bias=True)
        self.W_hh = nn.Linear(hidden_size, hidden_size, bias=True)

        # Output layer y_t = W_hy h_t + b_y
        self.fc_out = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        """
        x: [batch, seq_len, input_size]
        """
        batch_size, seq_len, _ = x.size()
        device = x.device

        # initial hidden state h_0
        h_t = torch.zeros(batch_size, self.hidden_size, device=device)

        outputs = []
        for t in range(seq_len):
            x_t = x[:, t, :]  # [batch, input_size] at time t

            # recurrent update
            h_t = torch.tanh(self.W_xh(x_t) + self.W_hh(h_t))  # [batch, hidden_size]

            # output at time t
            y_t = self.fc_out(h_t)  # [batch, output_size]
            outputs.append(y_t.unsqueeze(1))  # keep time dimension

        # concatenate along time dimension
        y = torch.cat(outputs, dim=1)  # [batch, seq_len, output_size]
        return y
```

RNN Example: Character-level Language Model

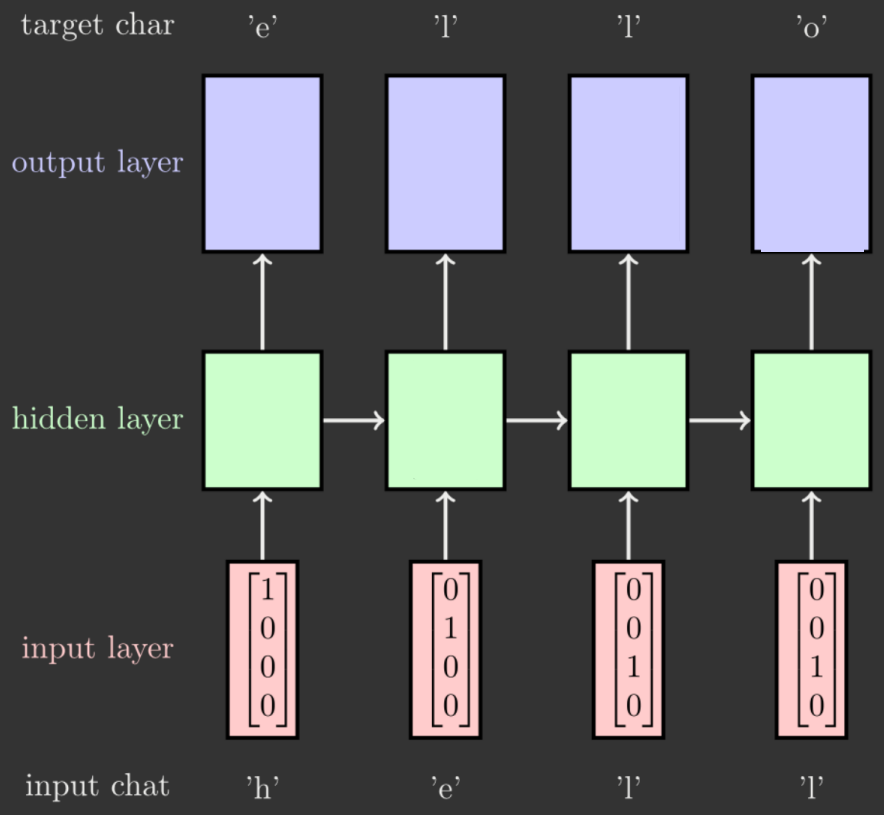
Given characters at {1,2,…,t},
model predicts character t
Training Sequence: “hello”; Vocabulary: [h,e,l,o]

$$h_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h, \qquad h_0 = 0$$
$$y_t = W_{hy}h_t + b_y.$$

$$W_{xh} = \begin{bmatrix} 0.300 & 0.867 & -0.344 & 0.000 \\ -0.100 & 0.767 & 1.056 & 0.000 \\ 0.900 & 0.433 & 0.811 & 0.000 \end{bmatrix}$$

$$W_{hh} = \begin{bmatrix} 0.444 & 0 & 0 \\ -1.556 & 0 & 0 \\ -1.111 & 0 & 0 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} 0.426 & -0.383 & 0.807 \\ 1.153 & -2.591 & 2.088 \\ -2.163 & 0.804 & -3.991 \\ 1.021 & -0.426 & 4.305 \end{bmatrix}, \quad b_y = \begin{bmatrix} 0.108 \\ -0.284 \\ 1.321 \\ -0.124 \end{bmatrix}$$



RNN Example: Character-level Language Model

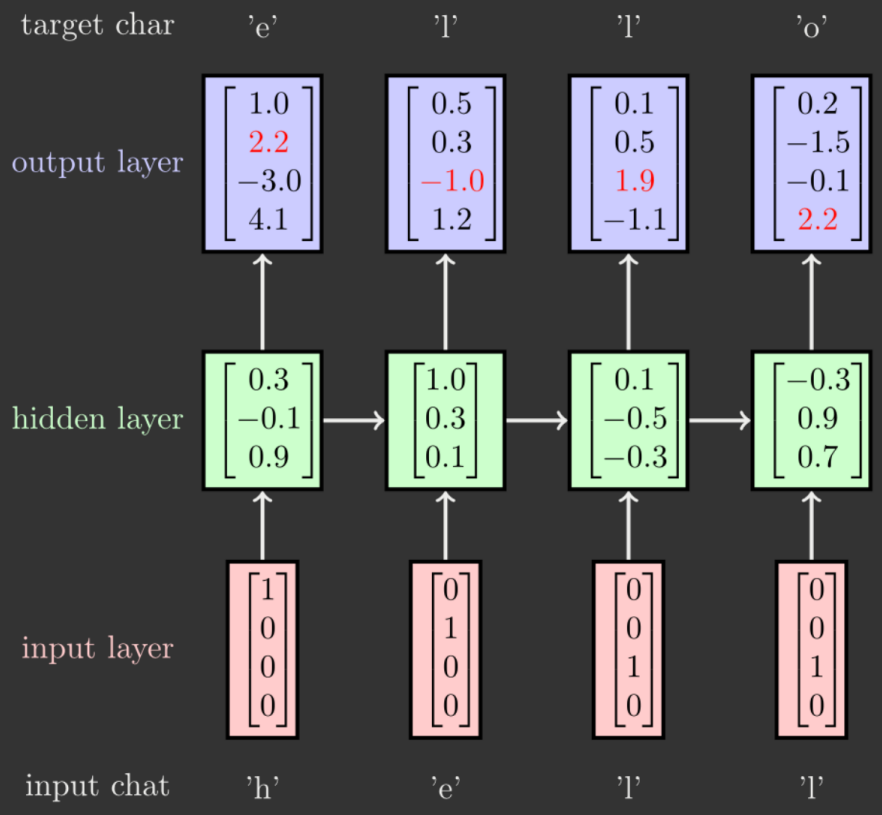
Given characters at {1,2,…,t},
model predicts character t
Training Sequence: “hello”; Vocabulary: [h,e,l,o]

$$h_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h, \qquad h_0 = 0$$
$$y_t = W_{hy}h_t + b_y.$$

$$W_{xh} = \begin{bmatrix} 0.300 & 0.867 & -0.344 & 0.000 \\ -0.100 & 0.767 & 1.056 & 0.000 \\ 0.900 & 0.433 & 0.811 & 0.000 \end{bmatrix}$$

$$W_{hh} = \begin{bmatrix} 0.444 & 0 & 0 \\ -1.556 & 0 & 0 \\ -1.111 & 0 & 0 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} 0.426 & -0.383 & 0.807 \\ 1.153 & -2.591 & 2.088 \\ -2.163 & 0.804 & -3.991 \\ 1.021 & -0.426 & 4.305 \end{bmatrix}, \quad b_y = \begin{bmatrix} 0.108 \\ -0.284 \\ 1.321 \\ -0.124 \end{bmatrix}$$

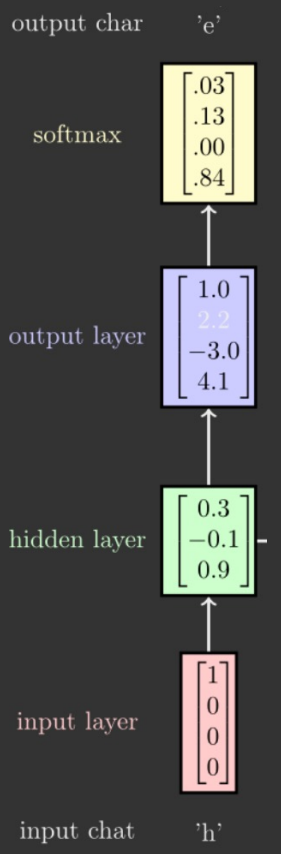


RNN Example: Character-level Language Model

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Vocabulary: [h,e,l,o]

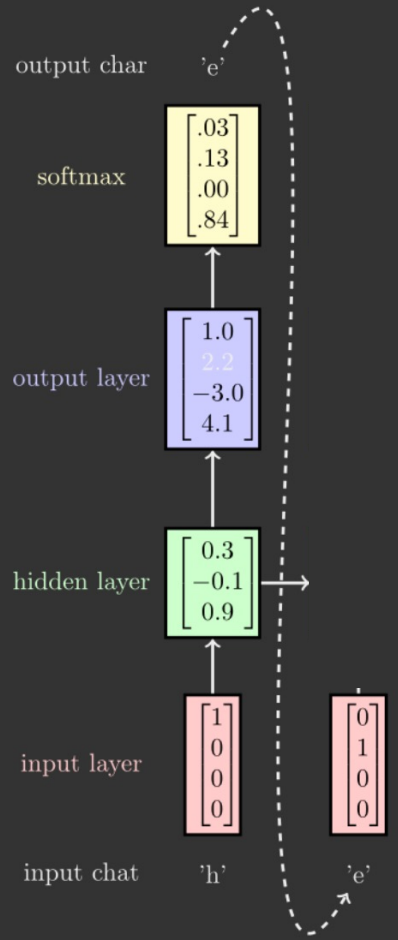
At test-time sample characters one at a time, feed back to model



RNN Example: Character-level Language Model

Vocabulary: [h,e,l,o]

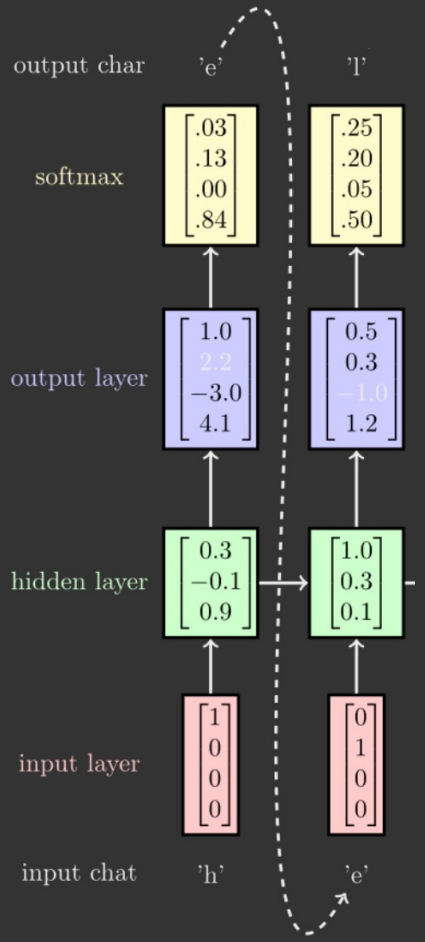
At test-time sample characters one at a time, feed back to model



RNN Example: Character-level Language Model

Vocabulary: [h,e,l,o]

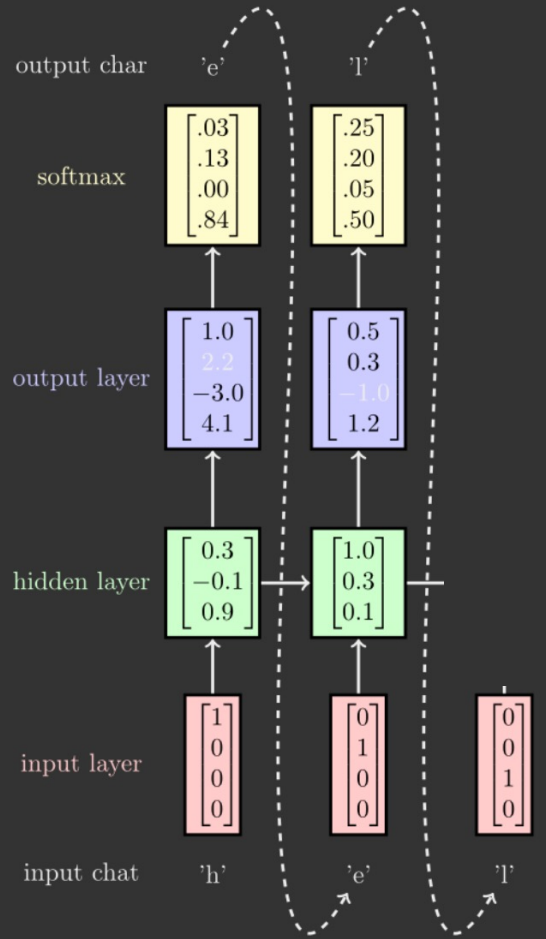
At test-time sample characters one at a time, feed back to model



RNN Example: Character-level Language Model

Vocabulary: [h,e,l,o]

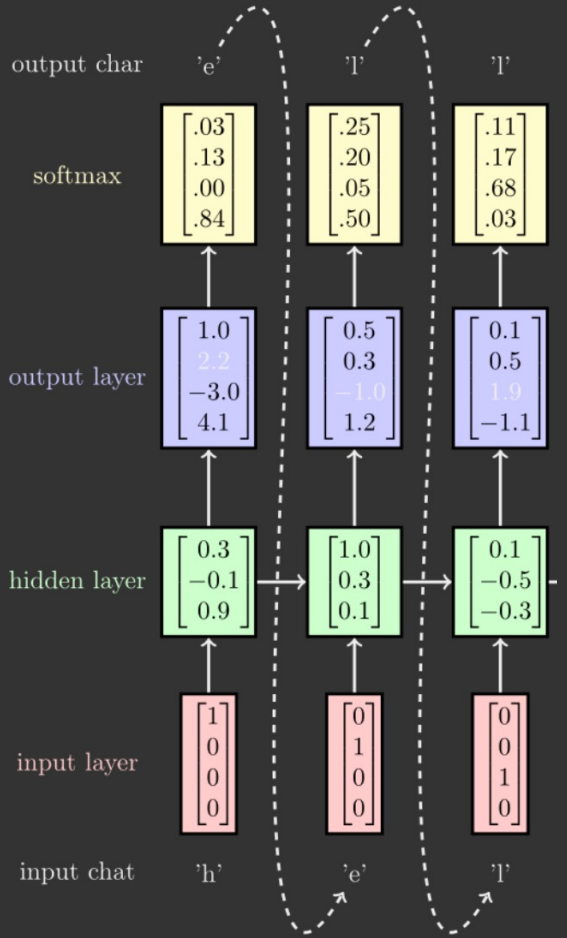
At test-time sample characters one at a time, feed back to model



RNN Example: Character-level Language Model

Vocabulary: [h,e,l,o]

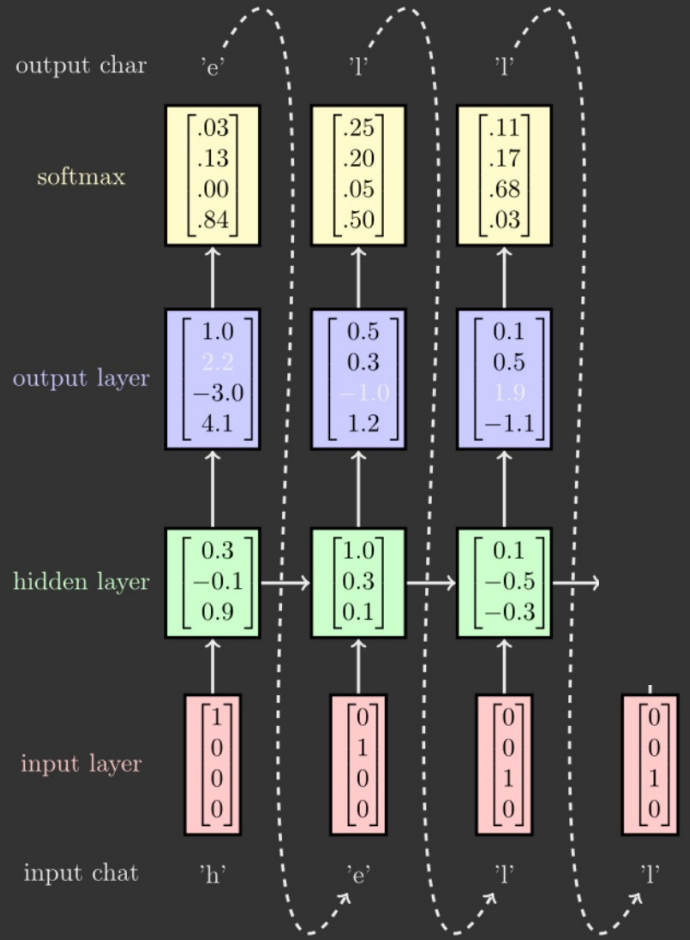
At test-time sample characters one at a time, feed back to model



RNN Example: Character-level Language Model

Vocabulary: [h,e,l,o]

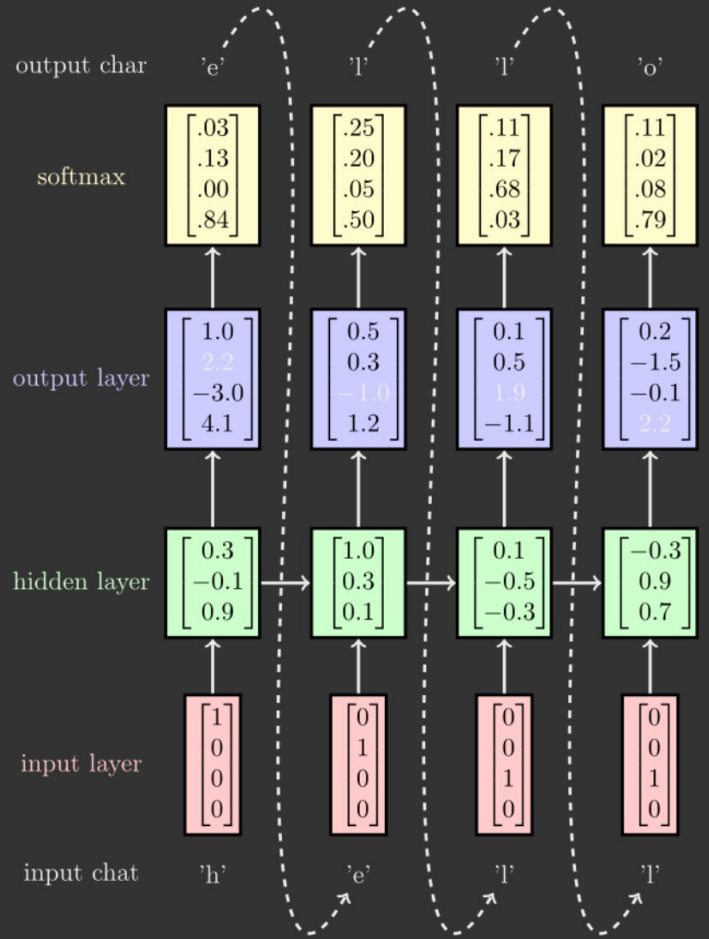
At test-time sample characters one at a time, feed back to model



RNN Example: Character-level Language Model

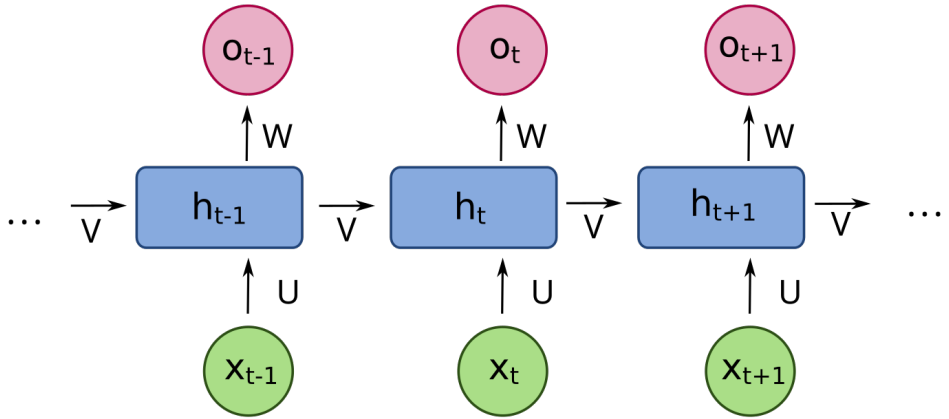
Vocabulary: [h,e,l,o]

At test-time sample characters one at a time, feed back to model



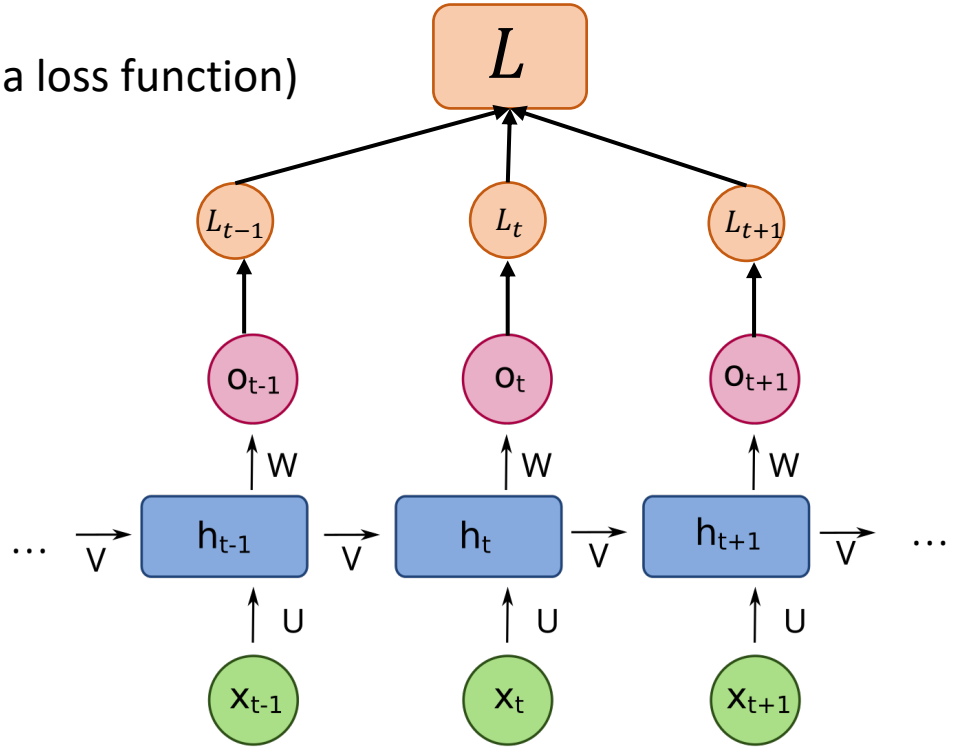
RNN: Training

1. Forward pass: make prediction



RNN: Training

1. Forward pass: make prediction
2. Compute prediction error (with respect to a loss function)

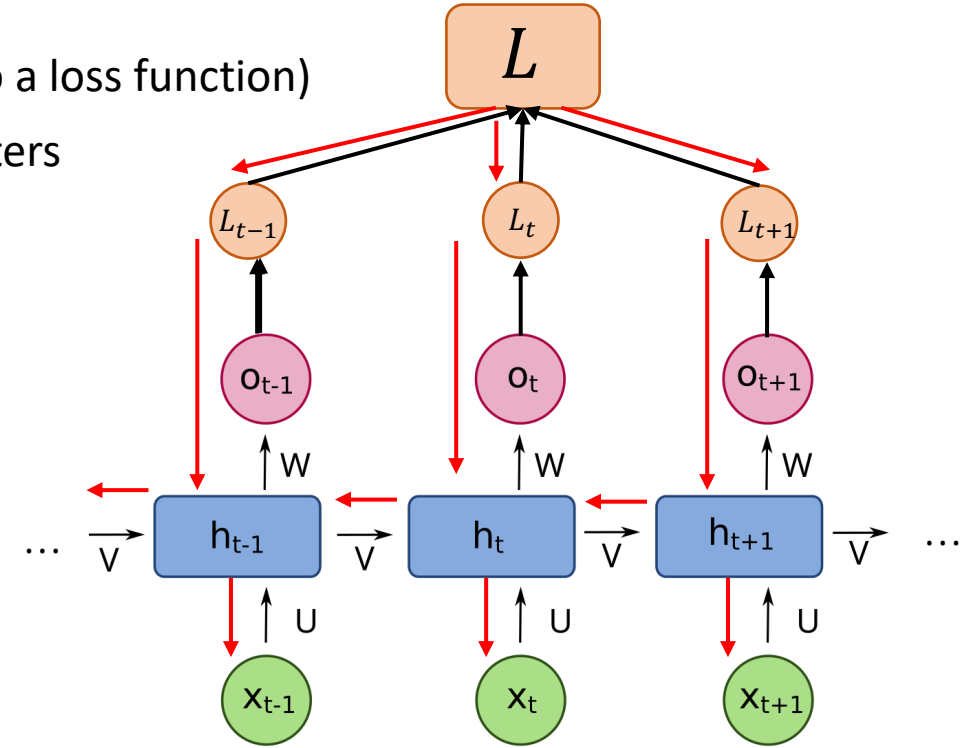


RNN: Training

1. Forward pass: make prediction
2. Compute prediction error (with respect to a loss function)
3. Backpropagate error to all model parameters
4. Update all model parameters

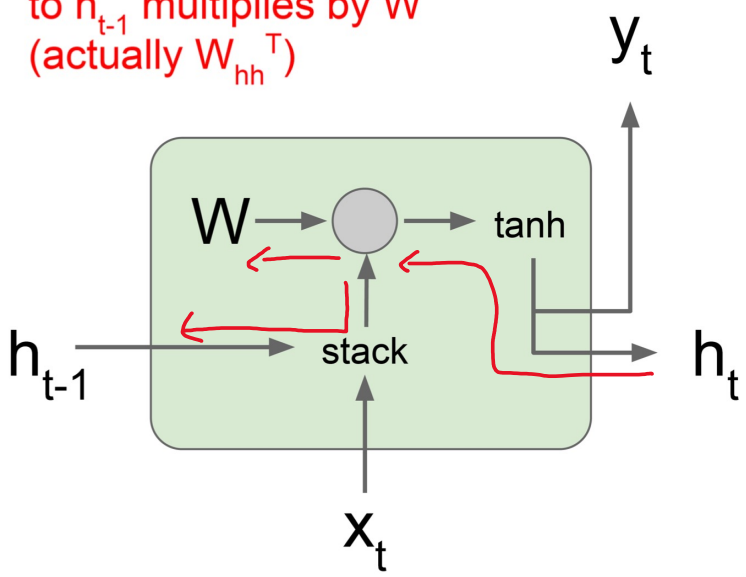
Backpropagation through time

which is the usual backpropagation process for weights and biases adapted to RNN



The Exploding/ Vanishing Gradient Problem

Backpropagation from h_t to h_{t-1} multiplies by W (actually W_{hh}^T)



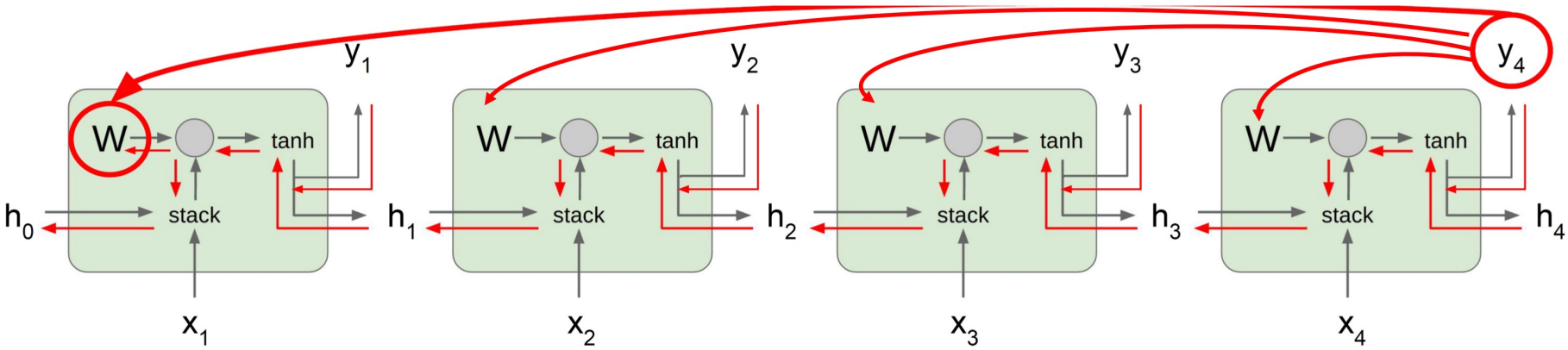
$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t)$$

$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

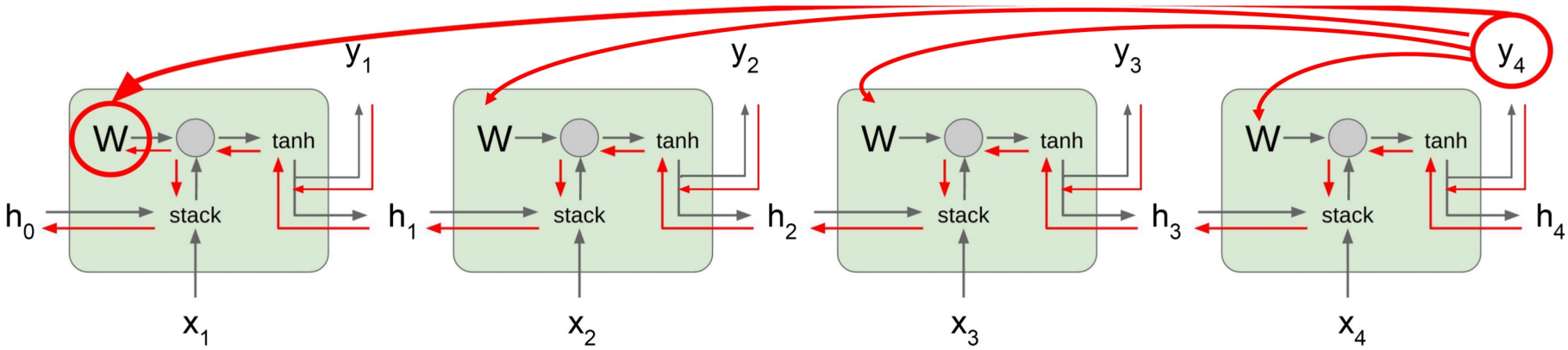
The Exploding/ Vanishing Gradient Problem



$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

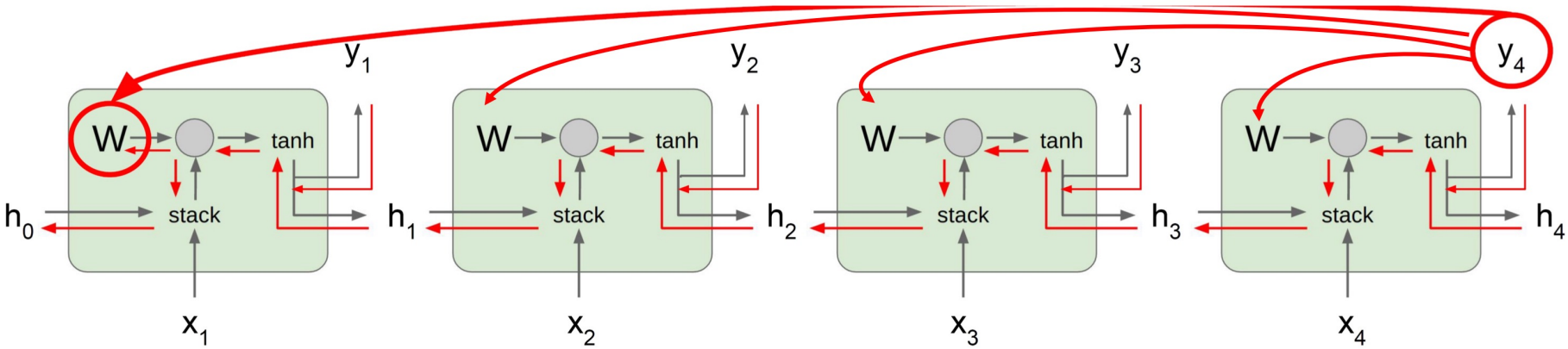
The Exploding/ Vanishing Gradient Problem



$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T}\left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}\right)\frac{\partial h_1}{\partial W}$$

- Vanishing gradients $\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\|_2 < 1$
- Exploding gradients $\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\|_2 > 1$

The Exploding/ Vanishing Gradient Problem

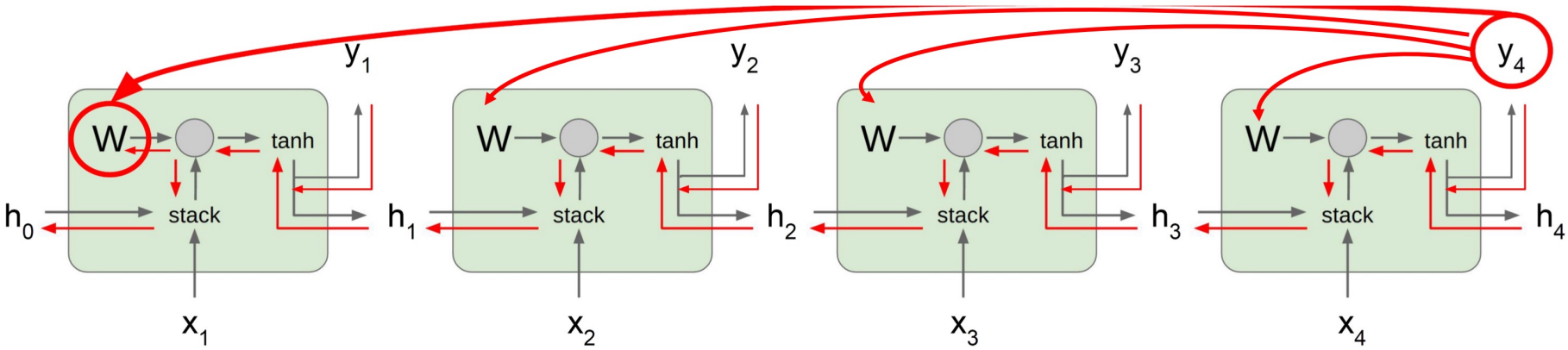


Many values >1:

Exploding Gradients

- Truncated Backpropagation
- Penalties
- Gradient Clipping

The Exploding/ Vanishing Gradient Problem



Many values >1:

Exploding Gradients

- Truncated Backpropagation
- Penalties
- Gradient Clipping

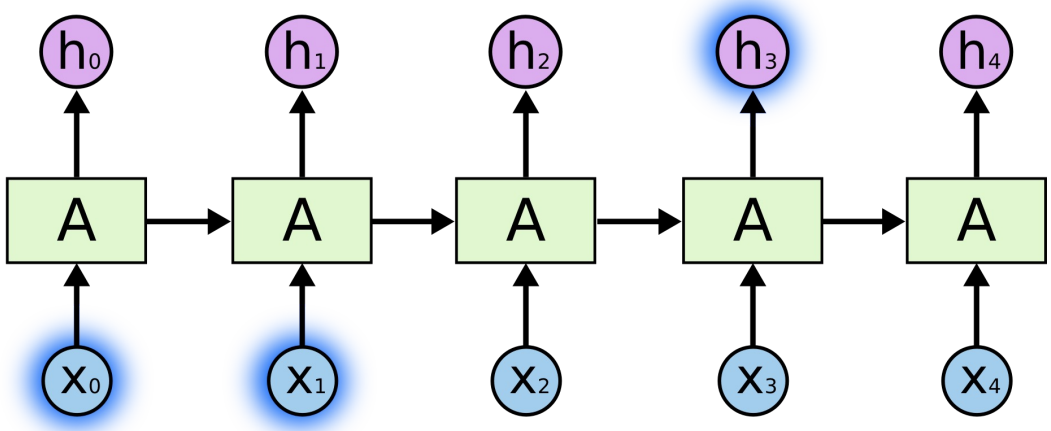
Many values <1:

Vanishing Gradients

- Activation function
- Weight initialization
- Network architecture

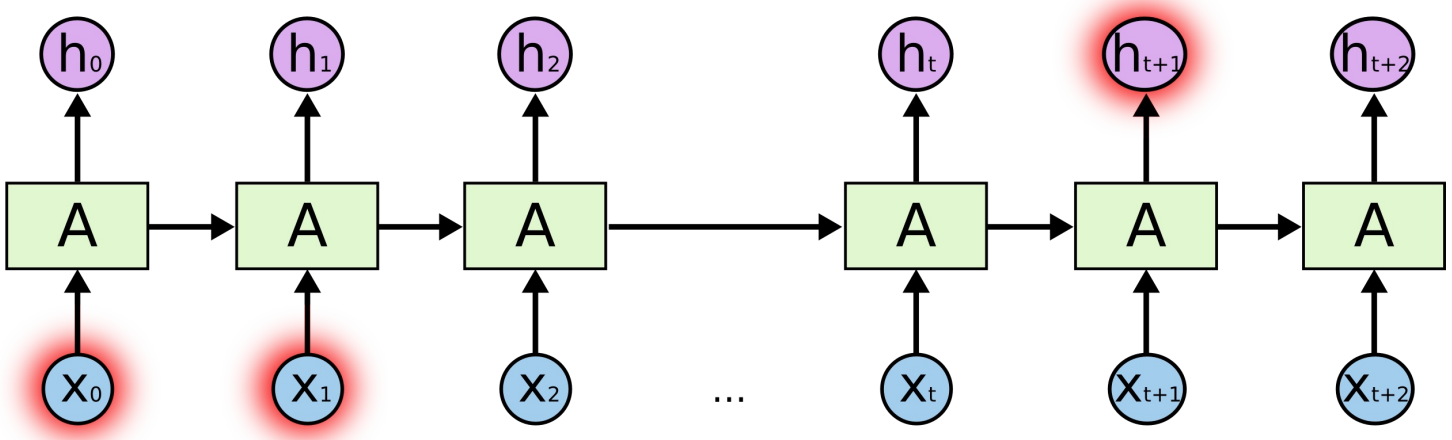
The Problem of Long-Term Dependencies

If we are trying to predict the last word in "**the clouds are in the *sky***," we don't need any further context – it's pretty obvious the next word is going to be sky.



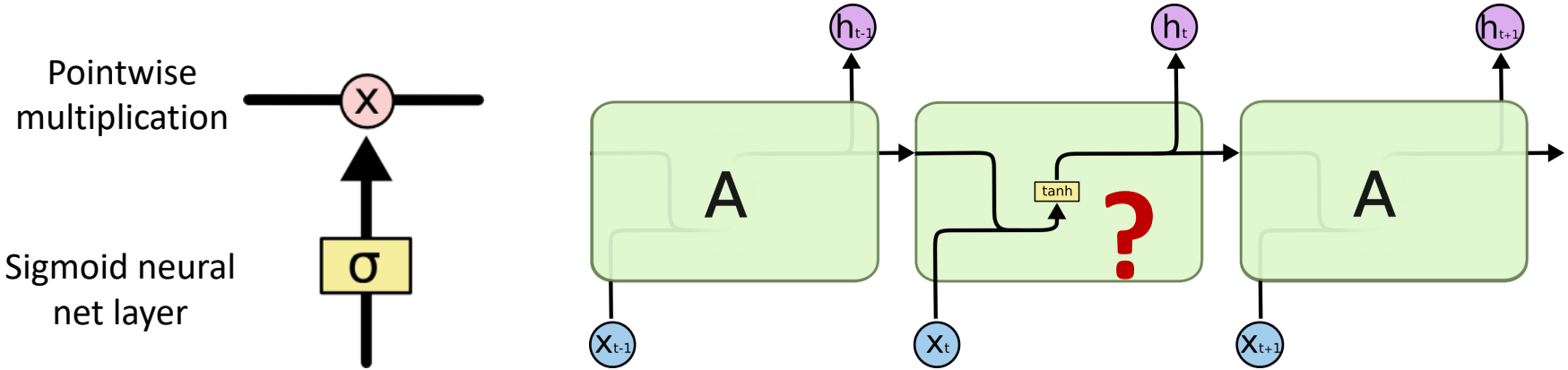
The Problem of Long-Term Dependencies

There are also cases where we need more context. Consider trying to predict the last word in the text "**I grew up in China… I speak fluent *Chinese***."



Gating Mechanisms in Neurons

Use gates to selectively **add** or **remove** information within recurrent unit get gated cell

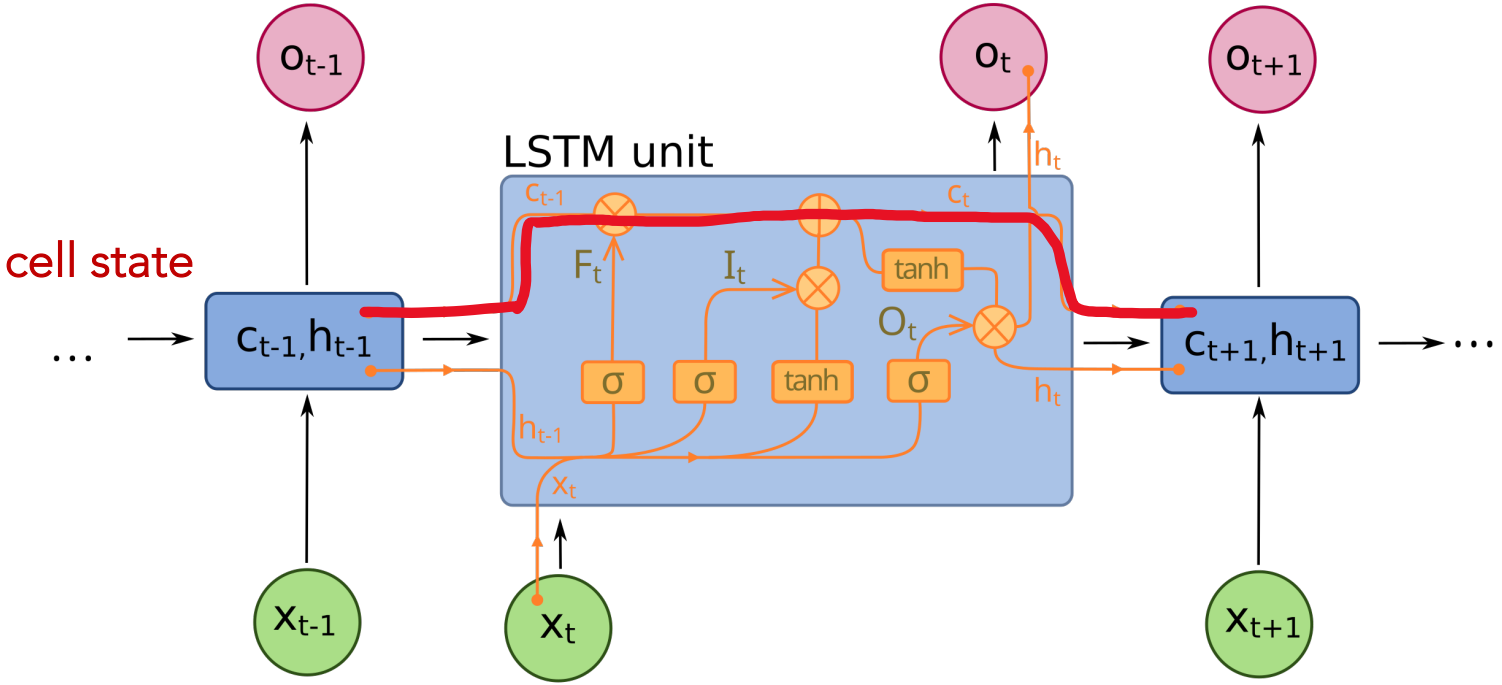


Gates are composed out of a sigmoid neural net layer and a pointwise multiplication operation

Long short-term memory (LSTM)

Hochreiter & Schmidhuber (1997)

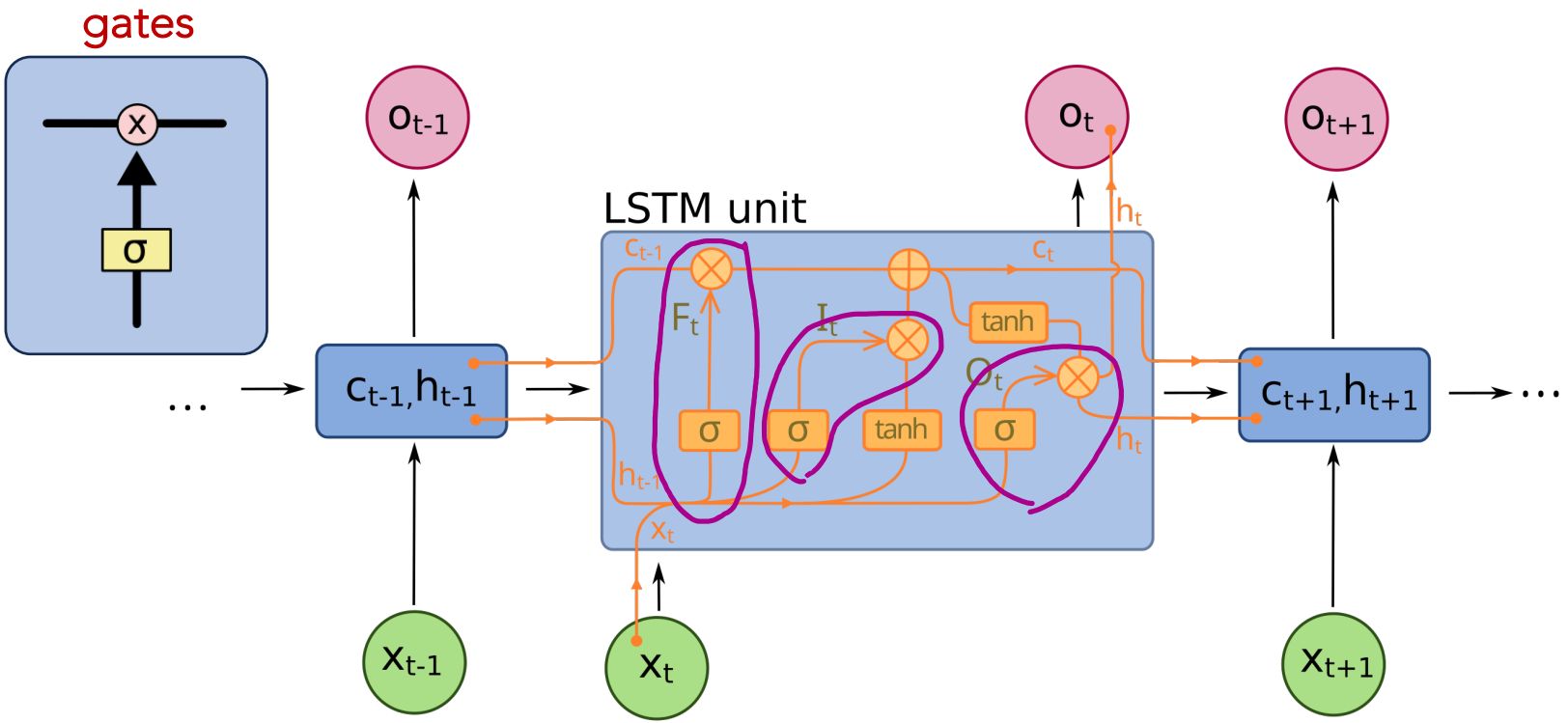
A type of RNN that is designed to better handle long-range dependencies



Long short-term memory (LSTM)

Hochreiter & Schmidhuber (1997)

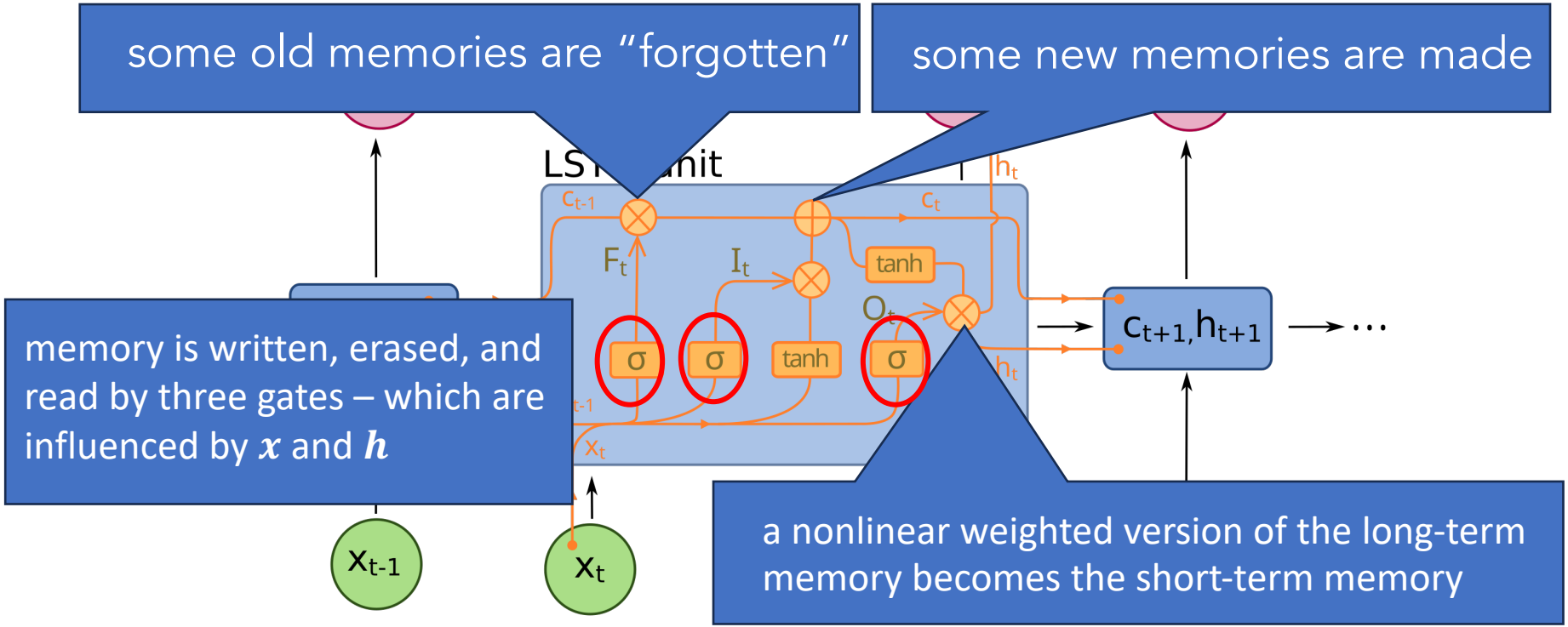
A type of RNN that is designed to better handle long-range dependencies



Long short-term memory (LSTM)

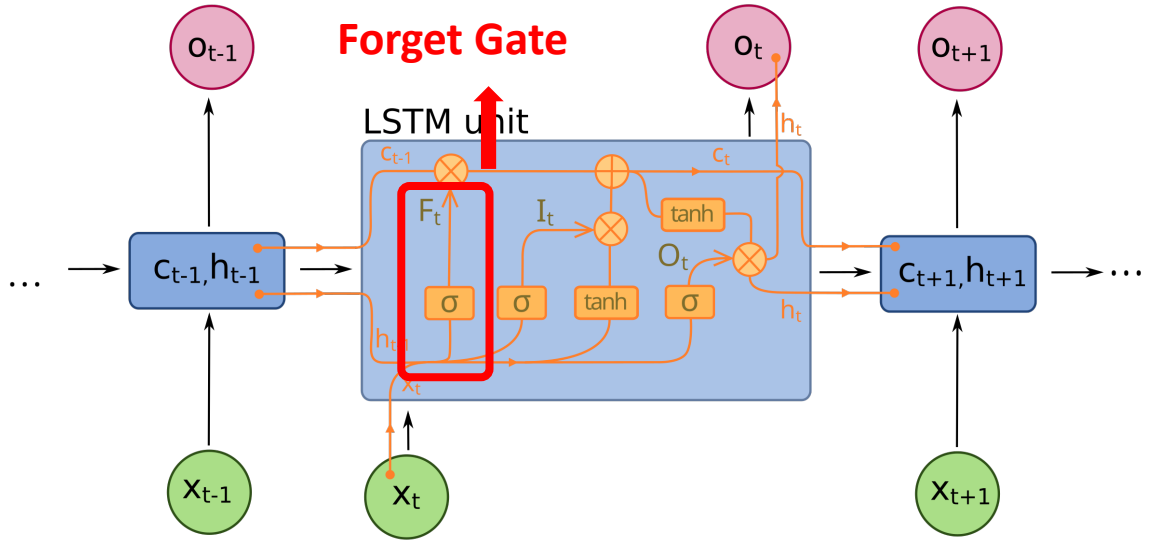
Hochreiter & Schmidhuber (1997)

A type of RNN that is designed to better handle long-range dependencies



Long short-term memory (LSTM)

Step1: Decide what information to throw away from the cell state, forget layer.

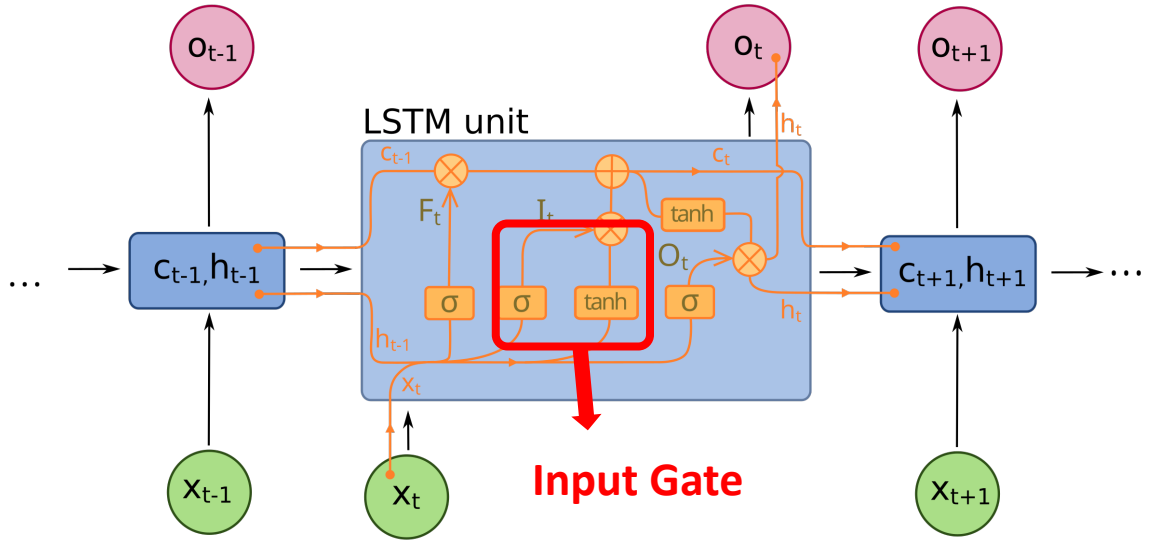


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

- 1 represents “completely keep this”
- 0 represents “completely get rid of this.”

Long short-term memory (LSTM)

Step 2: Decide what new information we're going to store in the cell state



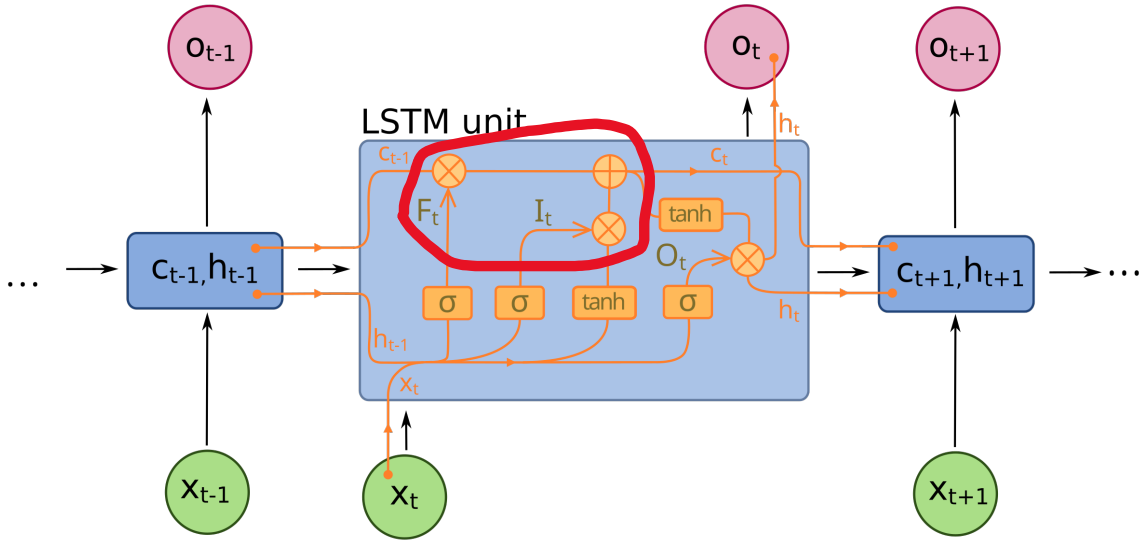
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

- Input gate layer : decides which values we will update
- Tanh layer : creates a vector of new candidate values

Long short-term memory (LSTM)

Step 3: Update the cell state

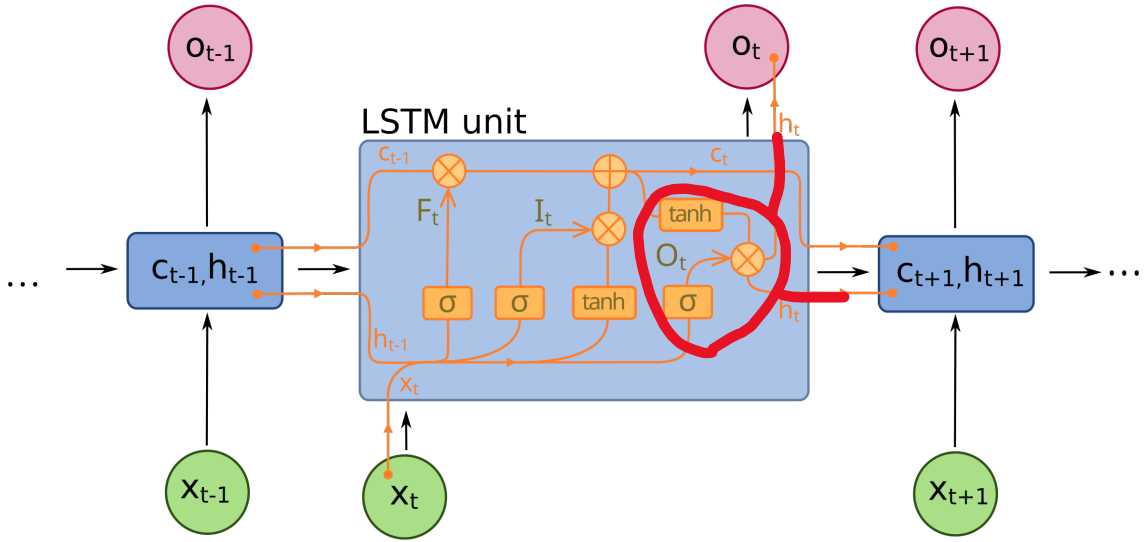


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

The cell state forgets some info, then it's simultaneously updated by us adding to it.

Long short-term memory (LSTM)

Step 4: Decide what the output/new hidden representation will be



Based on:
- filtered version of the cell state, and
- weighted version of recurrent, hidden layer

$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

```python
class LSTMScratch(d2l.Module):
    def __init__(self, num_inputs, num_hiddens, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()

        init_weight = lambda *shape: nn.Parameter(torch.randn(*shape) * sigma)
        triple = lambda: (init_weight(num_inputs, num_hiddens),
                          init_weight(num_hiddens, num_hiddens),
                          nn.Parameter(torch.zeros(num_hiddens)))
        self.W_xi, self.W_hi, self.b_i = triple()  # Input gate
        self.W_xf, self.W_hf, self.b_f = triple()  # Forget gate
        self.W_xo, self.W_ho, self.b_o = triple()  # Output gate
        self.W_xc, self.W_hc, self.b_c = triple()  # Input node
```

$$
\begin{aligned}
\mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{\text{xi}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hi}} + \mathbf{b}_{\text{i}}),\\
\mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{\text{xf}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hf}} + \mathbf{b}_{\text{f}}),\\
\mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{\text{xo}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{ho}} + \mathbf{b}_{\text{o}}),\\
\tilde{\mathbf{C}}_t &= \tanh(\mathbf{X}_t \mathbf{W}_{\text{xc}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hc}} + \mathbf{b}_{\text{c}}),\\
\mathbf{C}_t &= \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.\\
\mathbf{H}_t &= \mathbf{O}_t \odot \tanh(\mathbf{C}_t).
\end{aligned}
$$

```python
@d2l.add_to_class(LSTMScratch)
def forward(self, inputs, H_C=None):
    if H_C is None:
        # Initial state with shape: (batch_size, num_hiddens)
        H = torch.zeros((inputs.shape[1], self.num_hiddens),
                        device=inputs.device)
        C = torch.zeros((inputs.shape[1], self.num_hiddens),
                        device=inputs.device)
    else:
        H, C = H_C
    outputs = []
    for X in inputs:
        I = torch.sigmoid(torch.matmul(X, self.W_xi) +
                          torch.matmul(H, self.W_hi) + self.b_i)
        F = torch.sigmoid(torch.matmul(X, self.W_xf) +
                          torch.matmul(H, self.W_hf) + self.b_f)
        O = torch.sigmoid(torch.matmul(X, self.W_xo) +
                          torch.matmul(H, self.W_ho) + self.b_o)
        C_tilde = torch.tanh(torch.matmul(X, self.W_xc) +
                             torch.matmul(H, self.W_hc) + self.b_c)
        C = F * C + I * C_tilde
        H = O * torch.tanh(C)
        outputs.append(H)
    return outputs, (H, C)
```

```python
data = d2l.TimeMachine(batch_size=1024, num_steps=32)
lstm = LSTMScratch(num_inputs=len(data.vocab), num_hiddens=32)
model = d2l.RNNLMScratch(lstm, vocab_size=len(data.vocab), lr=4)
trainer = d2l.Trainer(max_epochs=50, gradient_clip_val=1, num_gpus=1)
trainer.fit(model, data)
```

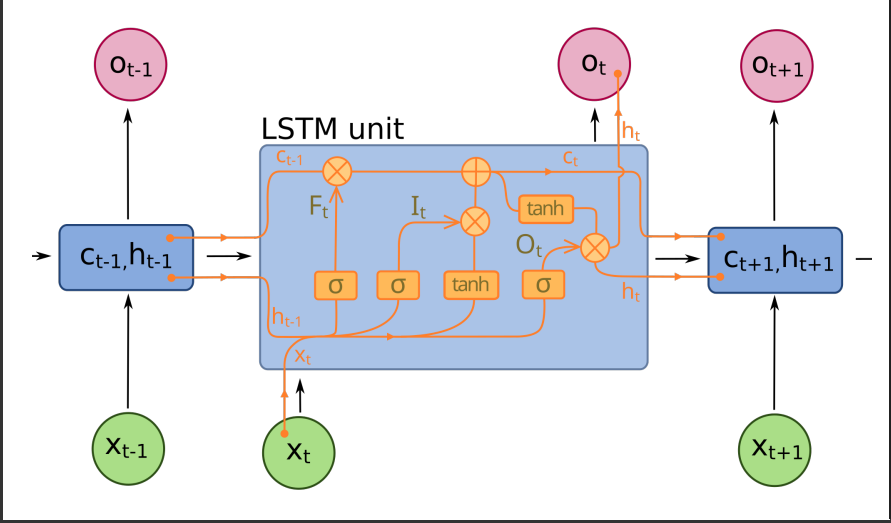
```python
class LSTM(d2l.RNN):
    def __init__(self, num_inputs, num_hiddens):
        d2l.Module.__init__(self)
        self.save_hyperparameters()
        self.rnn = nn.LSTM(num_inputs, num_hiddens)

    def forward(self, inputs, H_C=None):
        return self.rnn(inputs, H_C)

lstm = LSTM(num_inputs=len(data.vocab), num_hiddens=32)
model = d2l.RNNLM(lstm, vocab_size=len(data.vocab), lr=4)
trainer.fit(model, data)
```

https://d2l.ai/chapter_recurrent-modern/lstm.html

Long short-term memory (LSTM)



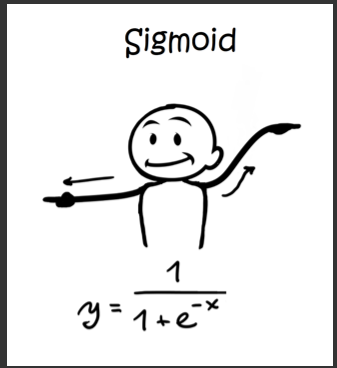
Case 1 $x_1 = 1, \quad h_0 = 0, \quad c_0 = 0$

$w_f = 0.0, \quad w_i = 1.0, \quad w_c = 1.0, \quad w_o = 1.0$

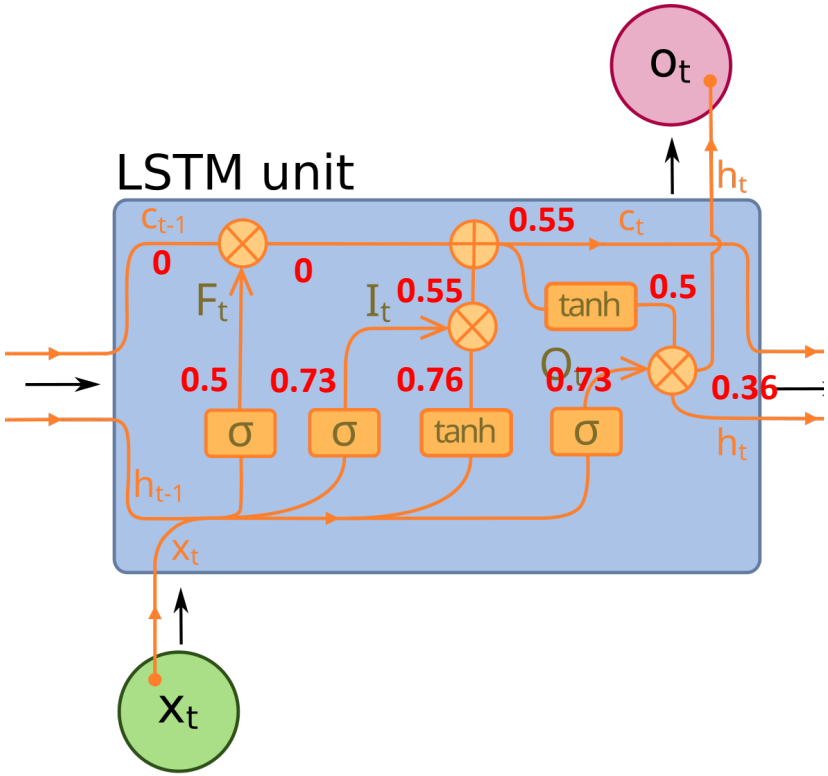
Case 2 $x_2 = 1, \quad h_1 = 0.36, \quad c_1 = 0.55$

$w_f = 2.0, \quad w_i = -2.0, \quad w_c = 1.0, \quad w_o = 1.0$

$$f_t = \sigma(w_f^x x_t + w_f^h h_{t-1} + b_f) \quad \text{(forget gate)}$$
$$i_t = \sigma(w_i^x x_t + w_i^h h_{t-1} + b_i) \quad \text{(input gate)}$$
$$\tilde{c}_t = \tanh(w_c^x x_t + w_c^h h_{t-1} + b_c) \quad \text{(candidate cell)}$$
$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \quad \text{(new cell state)}$$
$$o_t = \sigma(w_o^x x_t + w_o^h h_{t-1} + b_o) \quad \text{(output gate)}$$
$$h_t = o_t \cdot \tanh(c_t) \quad \text{(hidden state)}$$



Long short-term memory (LSTM)



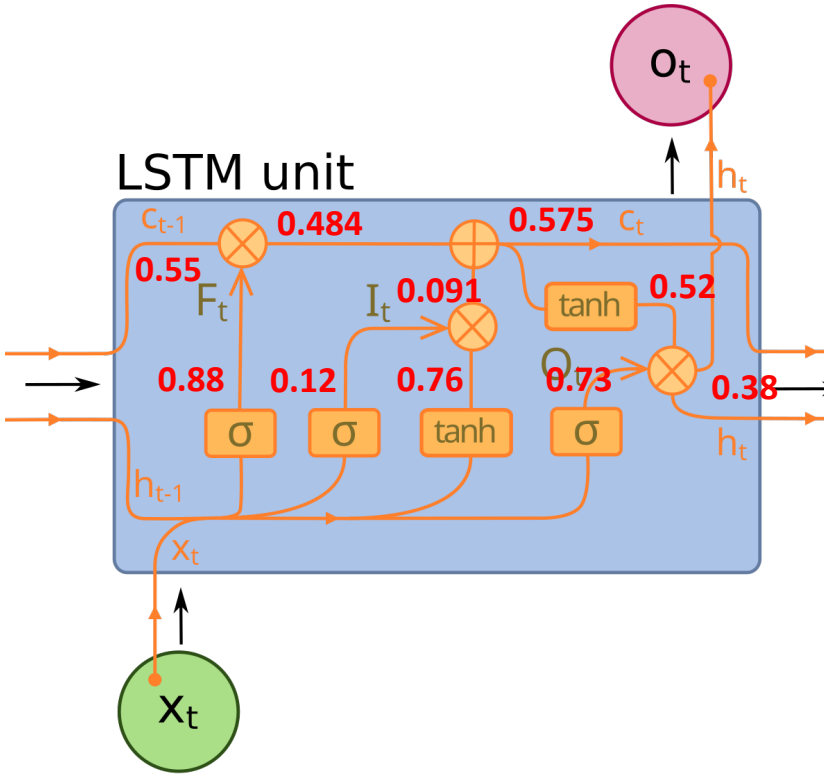
Case 1 $x_1 = 1, \quad h_0 = 0, \quad c_0 = 0$

$w_f = 0.0, \quad w_i = 1.0, \quad w_c = 1.0, \quad w_o = 1.0$

write new information into memory

$$f_t = \sigma(w_f^x x_t + w_f^h h_{t-1} + b_f) \quad \text{(forget gate)}$$
$$i_t = \sigma(w_i^x x_t + w_i^h h_{t-1} + b_i) \quad \text{(input gate)}$$
$$\tilde{c}_t = \tanh(w_c^x x_t + w_c^h h_{t-1} + b_c) \quad \text{(candidate cell)}$$
$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \quad \text{(new cell state)}$$
$$o_t = \sigma(w_o^x x_t + w_o^h h_{t-1} + b_o) \quad \text{(output gate)}$$
$$h_t = o_t \cdot \tanh(c_t) \quad \text{(hidden state)}$$

Long short-term memory (LSTM)



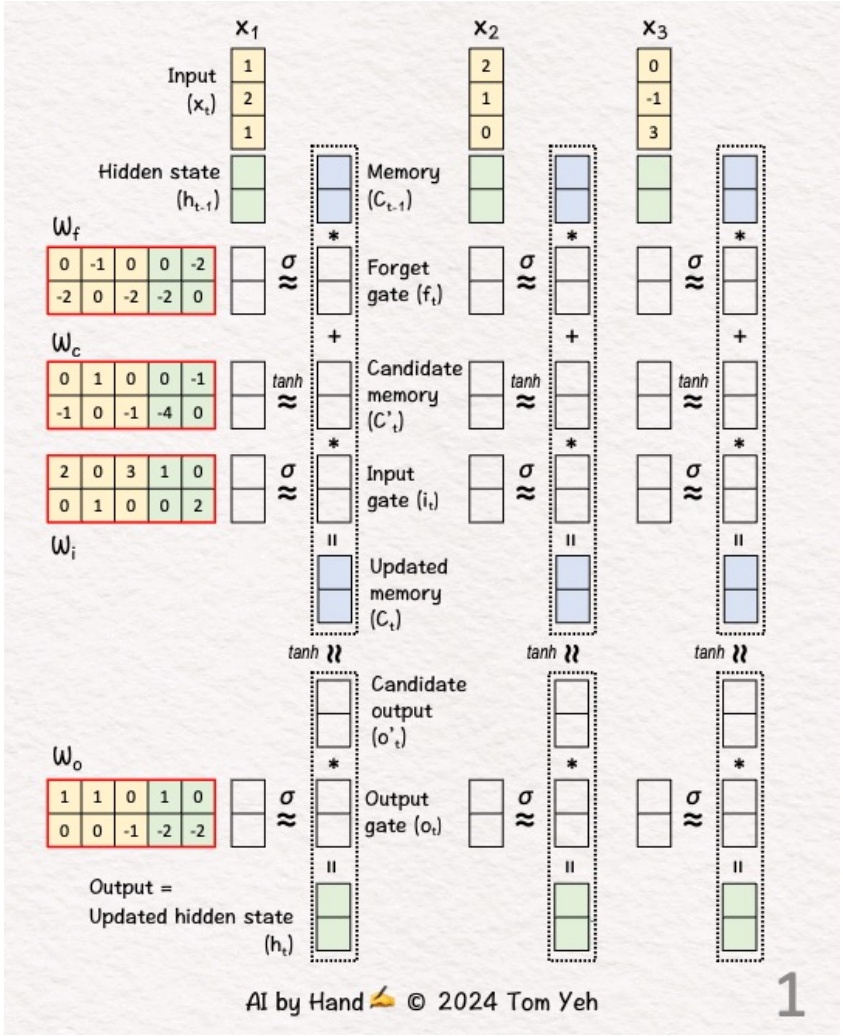
Case 2 $x_2 = 1, \quad h_1 = 0.36, \quad c_1 = 0.55$

$w_f = 2.0, \quad w_i = -2.0, \quad w_c = 1.0, \quad w_o = 1.0$

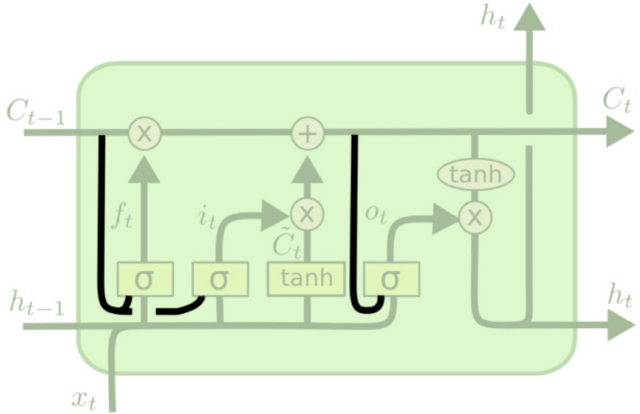
Try to keep old information

$$f_t = \sigma(w_f^x x_t + w_f^h h_{t-1} + b_f) \quad \text{(forget gate)}$$
$$i_t = \sigma(w_i^x x_t + w_i^h h_{t-1} + b_i) \quad \text{(input gate)}$$
$$\tilde{c}_t = \tanh(w_c^x x_t + w_c^h h_{t-1} + b_c) \quad \text{(candidate cell)}$$
$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \quad \text{(new cell state)}$$
$$o_t = \sigma(w_o^x x_t + w_o^h h_{t-1} + b_o) \quad \text{(output gate)}$$
$$h_t = o_t \cdot \tanh(c_t) \quad \text{(hidden state)}$$

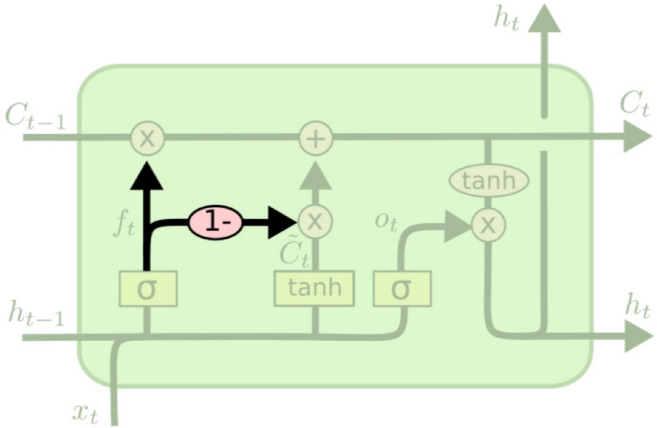
$$
\begin{aligned}
f_t &= \sigma(w_f^x x_t + w_f^h h_{t-1} + b_f) \quad \text{(forget gate)} \\
i_t &= \sigma(w_i^x x_t + w_i^h h_{t-1} + b_i) \quad \text{(input gate)} \\
\tilde{c}_t &= \tanh(w_c^x x_t + w_c^h h_{t-1} + b_c) \quad \text{(candidate cell)} \\
c_t &= f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \quad \text{(new cell state)} \\
o_t &= \sigma(w_o^x x_t + w_o^h h_{t-1} + b_o) \quad \text{(output gate)} \\
h_t &= o_t \cdot \tanh(c_t) \quad \text{(hidden state)}
\end{aligned}
$$



Variants of LSTM

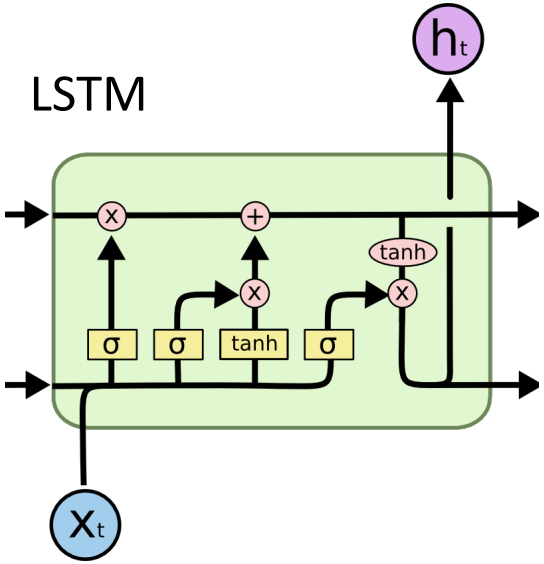


$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

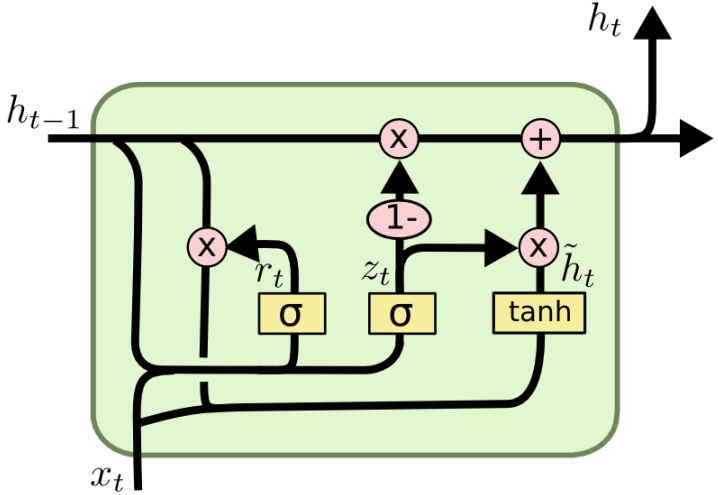


$$C_t = f_t * C_{t-1} + (\mathbf{1} - \boldsymbol{f_t}) * \tilde{C}_t$$

Gated Recurrent Units



No additional cell state!
Memory operates only and directly via the hidden state



Update of the hidden state can be divided into four steps:

1. **Reset Gate**: influence of the previous hidden state
2. **Update Gate**: influence of a newly computed update
3. Proposing an updated hidden state
4. Computing updated hidden state

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

```python
class GRUScratch(d2l.Module):
    def __init__(self, num_inputs, num_hiddens, sigma=0.01):
        super().__init__()
        self.save_hyperparameters()

        init_weight = lambda *shape: nn.Parameter(torch.randn(*shape) * sigma)
        triple = lambda: (init_weight(num_inputs, num_hiddens),
                          init_weight(num_hiddens, num_hiddens),
                          nn.Parameter(torch.zeros(num_hiddens)))
        self.W_xz, self.W_hz, self.b_z = triple()  # Update gate
        self.W_xr, self.W_hr, self.b_r = triple()  # Reset gate
        self.W_xh, self.W_hh, self.b_h = triple()  # Candidate hidden state

@d2l.add_to_class(GRUScratch)
def forward(self, inputs, H=None):
    if H is None:
        # Initial state with shape: (batch_size, num_hiddens)
        H = torch.zeros((inputs.shape[1], self.num_hiddens),
                        device=inputs.device)
    outputs = []
    for X in inputs:
        Z = torch.sigmoid(torch.matmul(X, self.W_xz) +
                          torch.matmul(H, self.W_hz) + self.b_z)
        R = torch.sigmoid(torch.matmul(X, self.W_xr) +
                          torch.matmul(H, self.W_hr) + self.b_r)
        H_tilde = torch.tanh(torch.matmul(X, self.W_xh) +
                             torch.matmul(R * H, self.W_hh) + self.b_h)
        H = Z * H + (1 - Z) * H_tilde
        outputs.append(H)
    return outputs, H
```

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{\text{xr}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hr}} + \mathbf{b}_{\text{r}}),$$
$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{\text{xz}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hz}} + \mathbf{b}_{\text{z}}),$$

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{\text{xh}} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}}),$$

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t.$$

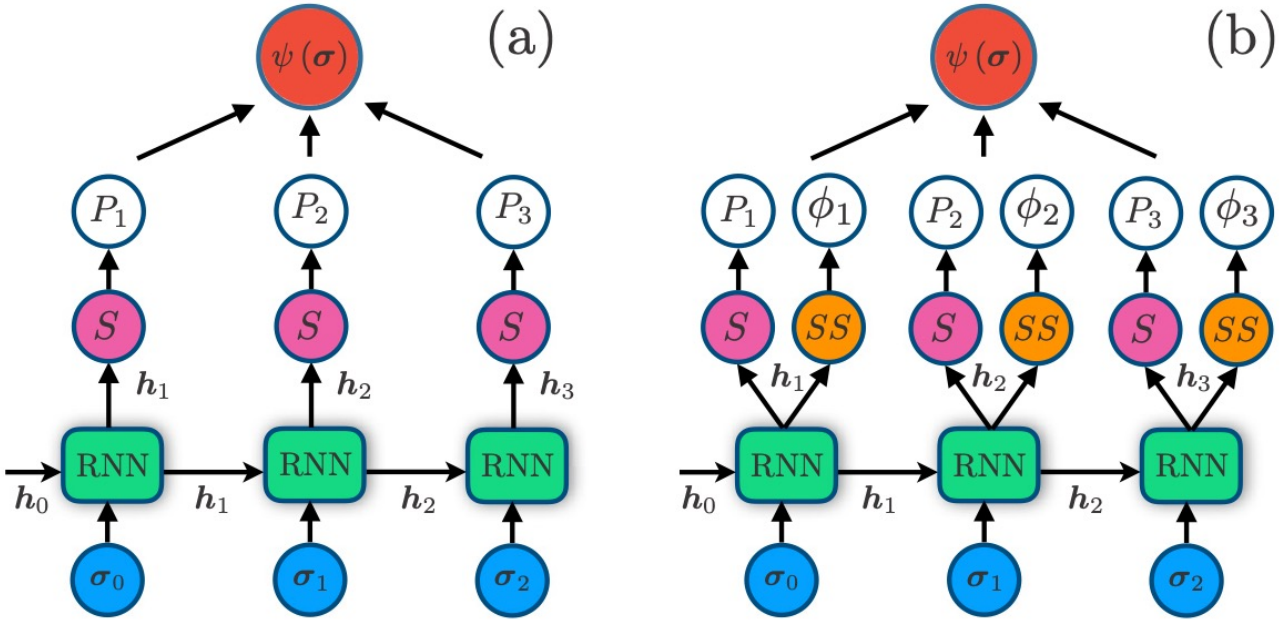
```python
class GRU(d2l.RNN):
    def __init__(self, num_inputs, num_hiddens):
        d2l.Module.__init__(self)
        self.save_hyperparameters()
        self.rnn = nn.GRU(num_inputs, num_hiddens)
```

Some Examples in (Quantum) Physics

Neural quantum states

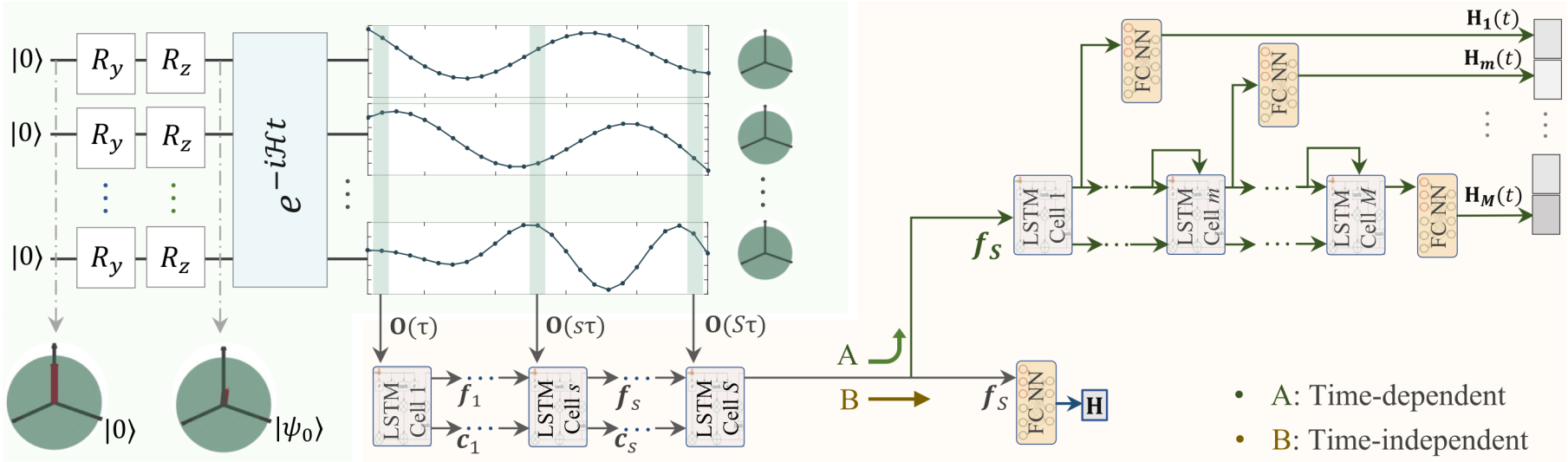
Phys. Rev. Research **2**, 023358 (2020)

$$|\Psi\rangle = \sum_{\boldsymbol{\sigma}} \psi(\boldsymbol{\sigma})\,|\boldsymbol{\sigma}\rangle = \sum_{\boldsymbol{\sigma}} \sqrt{P(\boldsymbol{\sigma})}\,|\boldsymbol{\sigma}\rangle \qquad |\Psi\rangle = \sum_{\boldsymbol{\sigma}} \exp[i\phi(\boldsymbol{\sigma})]\sqrt{P(\boldsymbol{\sigma})}\,|\boldsymbol{\sigma}\rangle.$$



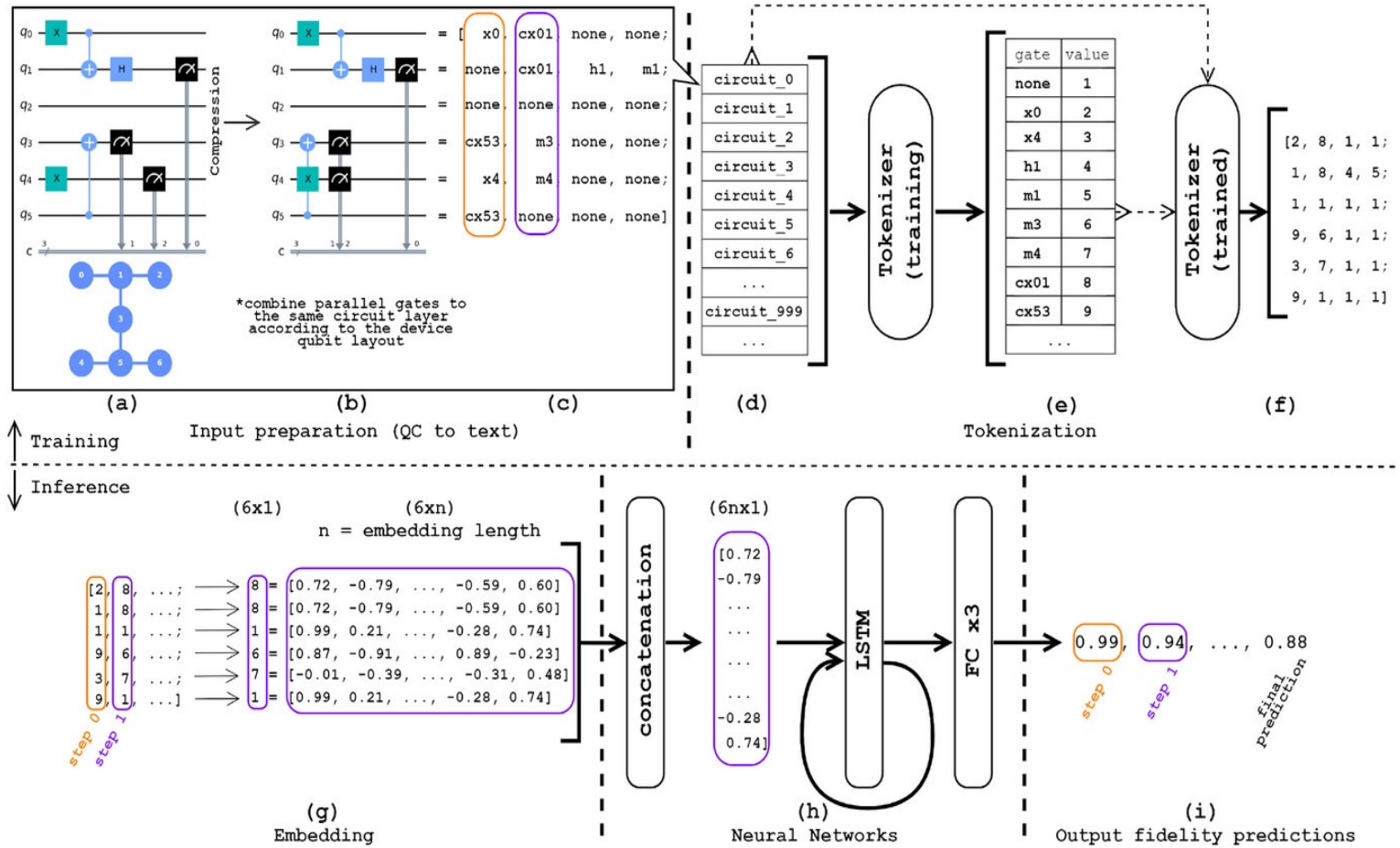
Learning quantum Hamiltonians

Phys. Rev. Research 3, 023246 (2021)



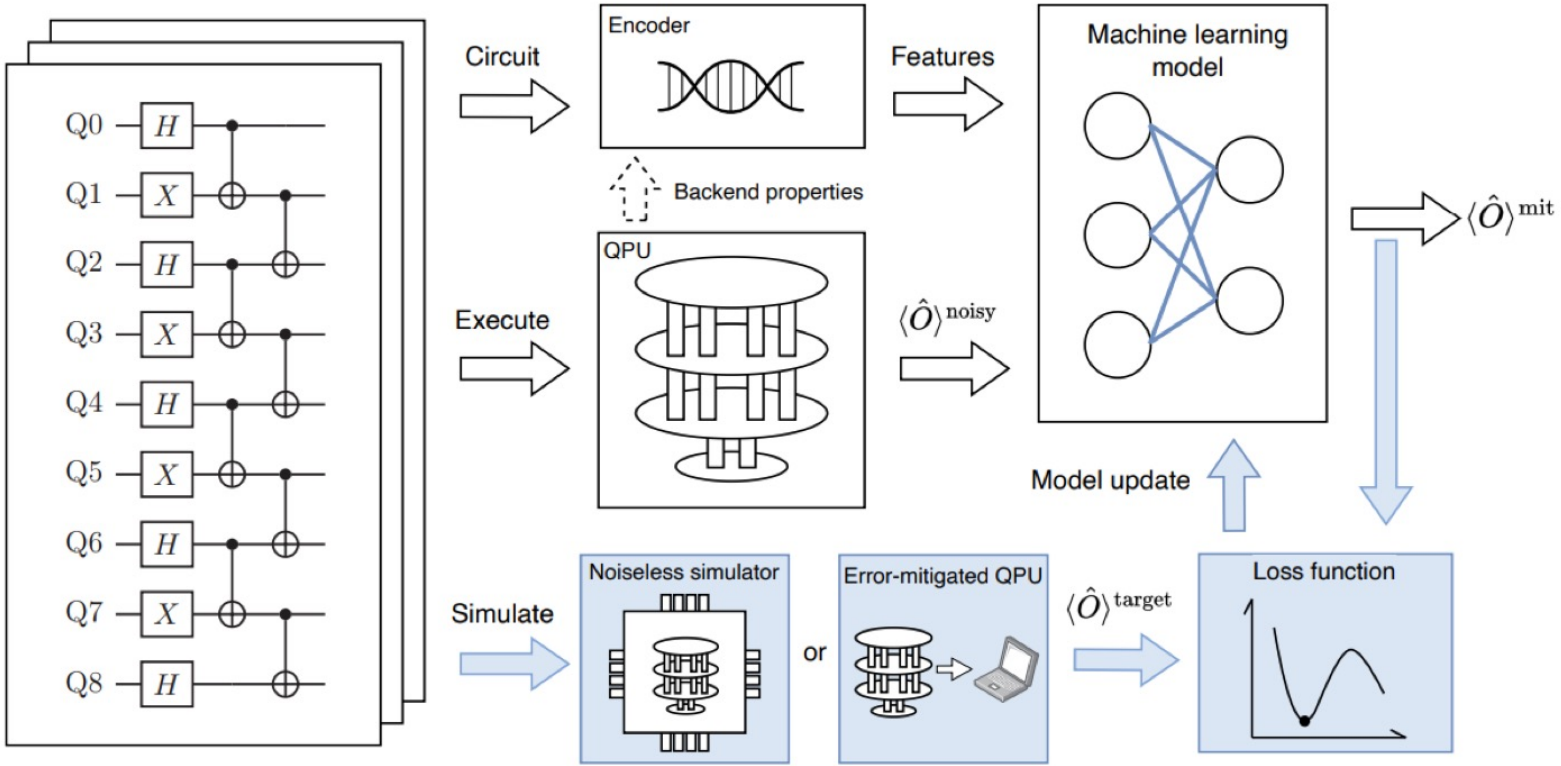
Quantum Circuit Fidelity Improvement

https://doi.org/10.1002/qute.202500022

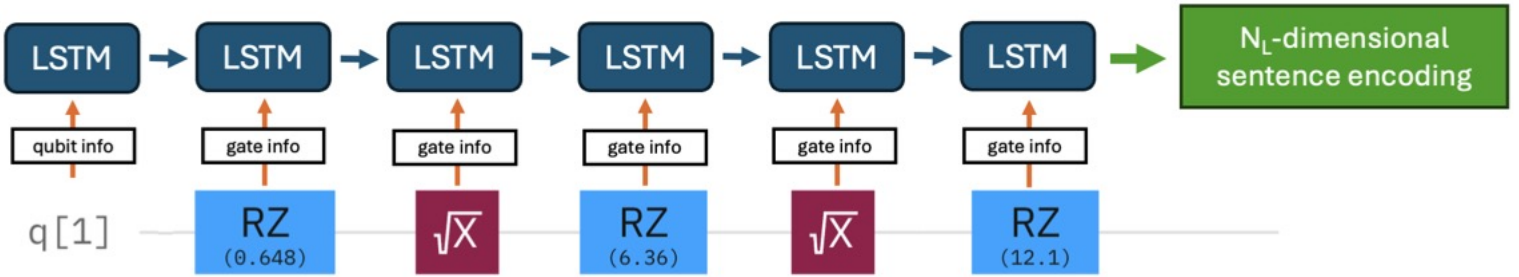


Quantum Error Mitigation with Machine Learning

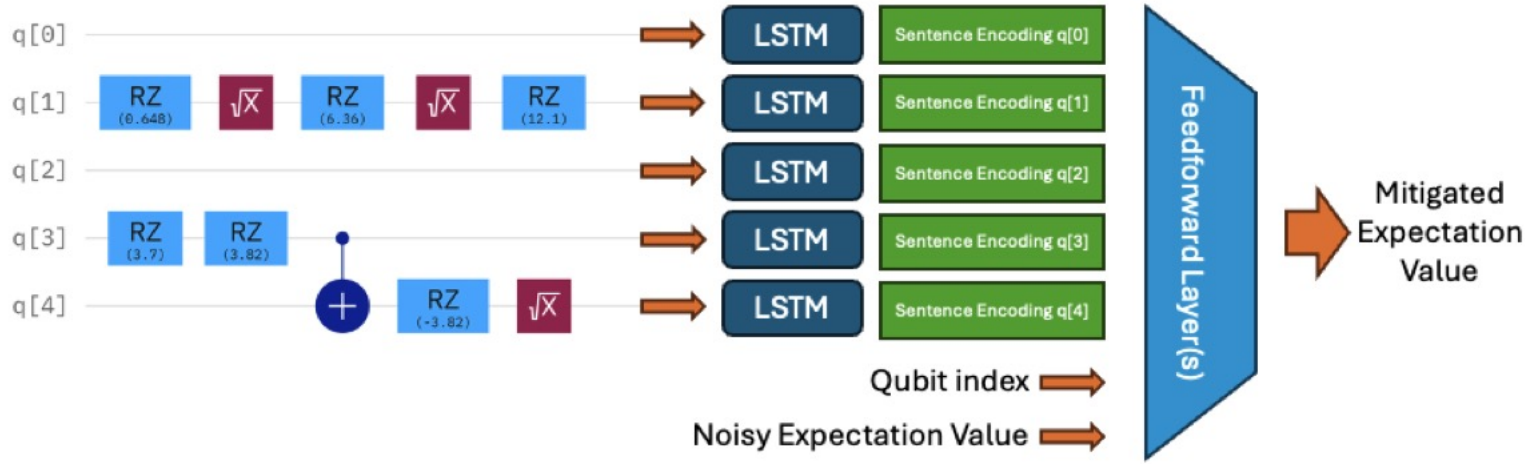
Nat Mach Intell **6**, 1478–1486 (2024), “Machine learning for practical quantum error mitigation”
Andrej Hulák, et al. “Quantum Error Mitigation with Deep Learning and Sequence Model”



Quantum Error Mitigation with Machine Learning



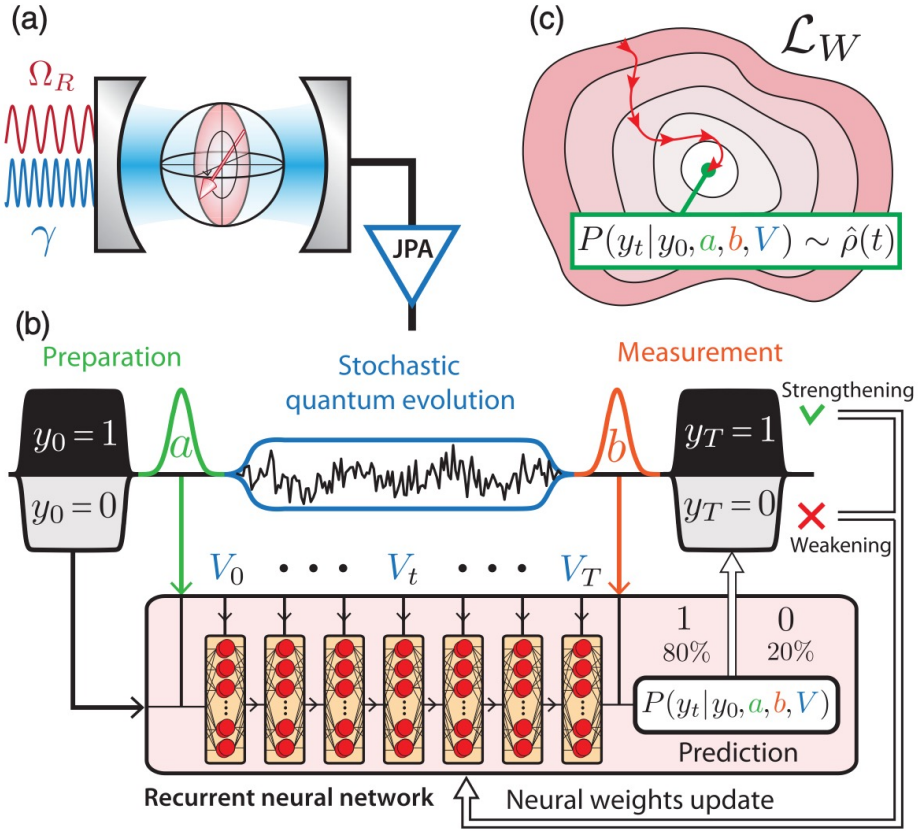
Quantum Circuits as Sequences



Reconstruct Quantum Dynamics of a Superconducting Qubit

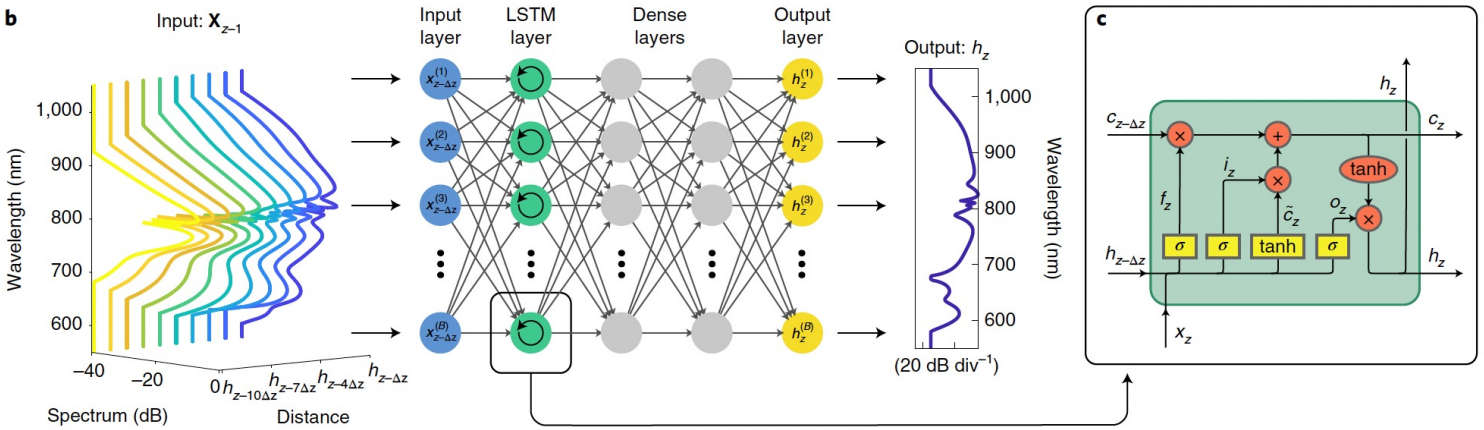
Phys. Rev. X **10**, 011006 (2020)

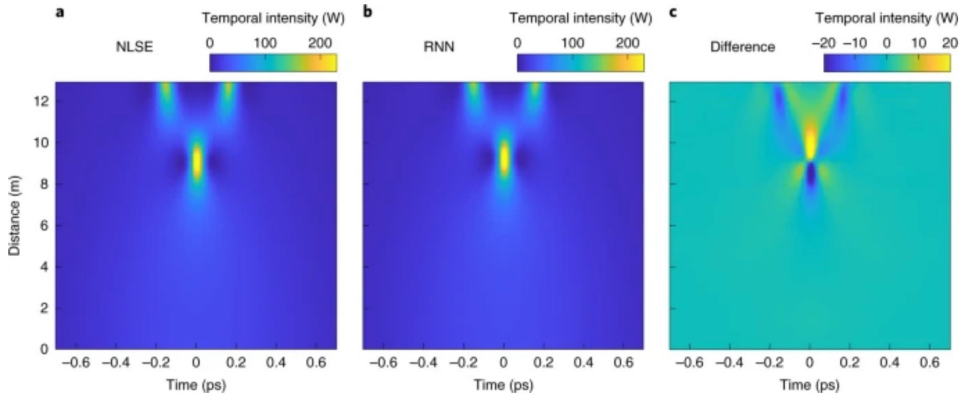
A model that extracts hidden correlations in vast datasets, can infer the complex quantum dynamics of a single superconducting qubit by supervised learning.



Predicting ultrafast nonlinear dynamics in fibre optics

Nat Mach Intell **3**, 344–354 (2021)





Question?